

מבוא למדעי המחשב

שיעור 2: סדרות

 Python » 3.4.1 Documentation » The Python Standard Library »

4. Built-in Types

- 4.6. Sequence Types — `list`, `tuple`, `range`
- 4.7. Text Sequence Type — `str`

לקריאה נוספת:

- http://he.wikipedia.org/wiki/ספרות_בבליות
- Dava Sobel, *Longitude - The True Story of a Lone Genius Who Solved the Greatest Scientific Problem of His Time*, Walker & Co., 2007.
- D. E. Knuth, The complexity of songs, *Communications of the Association of Computing Machinery*, 27(4):344-346, April 1984.

הצגת הנושא

- רוב טיפוסים האובייקטים שנתקלנו בהם עד כה (float, int, bool) הם סקלרים (scalar), כלומר, מחזיקים ערך בודד.
- לעיתים מזומנות אנחנו נתקלים בצורך לקבץ אובייקטים ל"מיכל" על מנת לבצע על כל האובייקטים במיכל אותו חישוב.
- הספרייה של פייתון מספקת מגוון מיכלים (containers) מסוגים שונים. אנחנו נדגים רק חלק מהאפשרויות.
- סוג מיכלים שימושי ביותר הוא סדרות (sequences). ראינו כבר שני נציגים לטיפוסים של סדרות: str ו-range.

סדרות

- בפייתון יש כמה טיפוסים של סדרות, למשל `str`, `range`, `list`, `tuple`. אנחנו נעבור על הטיפוסים הללו עוד מעט.
- המשותף לכולם הוא התכונות הבאות:
 - כל סדרה מכילה מספר סופי של אובייקטים.
 - האובייקטים מסודרים בסדר כלשהו.
 - אפשר לעבור על האובייקטים בסדרה לפי הסדר.
 - אפשר גם לבדוק אם אובייקט נמצא בסדרה.

הטיפוס range

- הטיפוס הזה מאפשר להגדיר סדרה חשבונית סופית של מספרים שלמים.

- דוגמאות:

<code>range(10)</code>	- הסדרה 0,1,2,3,4,5,6,7,8,9
<code>range(2,10)</code>	- הסדרה 2,3,4,5,6,7,8,9
<code>range(0,10,2)</code>	- הסדרה 0,2,4,6,8
<code>range(1,-5,-1)</code>	- הסדרה 1,0,-1,-2,-3,-4

- הטיפוס הזה שימושי במיוחד עבור לולאות `for`

שימוש ב-range

```
x = 7
```

```
x in range(10)
```

```
x in range(0,10,2)
```

```
prime = True
```

```
for i in range(2,x):
```

```
    if x%i == 0:
```

```
        prime = False
```

```
print(prime)
```

שימוש ב-range

```
x = 7
```

```
x in range(10) True
```

```
x in range(0,10,2)
```

```
prime = True
```

```
for i in range(2,x):
```

```
    if x%i == 0:
```

```
        prime = False
```

```
print(prime)
```

שימוש ב-range

```
x = 7
```

```
x in range(10) True
```

```
x in range(0,10,2) False
```

```
prime = True
```

```
for i in range(2,x):
```

```
    if x%i == 0:
```

```
        prime = False
```

```
print(prime)
```

שימוש ב-range

`x = 7`

`x in range(10)`

True

`x in range(0,10,2)`

False

`prime = True`

`for i in range(2,x):`

`if x%i == 0:`

`prime = False`

`print(prime)`

אפשר לבדוק רק עד $\sqrt{x}$
למה?

הטיפוס str

- אובייקט מטיפוס str מחזיק סדרה של תווים בייצוג unicode, מה שמאפשר לנו לייצג טקסט.

- דוגמאות:

קבוע str -

'the quick brown fox jumps over the lazy dog'
"the quick brown fox jumps over the lazy dog"
""the quick brown fox jumps over the lazy dog""
""the quick brown fox jumps over the lazy dog""

משתנה str - s_eng = "The rain in Spain stays mainly in the plain"

s_heb = "ברד ירד בדרום ספרד הערב"

שימוש ב-str

ביטוי	ערך
s_eng[:]	"The rain in Spain stays mainly in the plain"
s_eng[2]	"e"
s_eng[:2]	"Th"
s_eng[2:]	"e rain in Spain stays mainly in the plain"
s_eng[2:7]	"e rai"
s_eng[2:7:2]	"eri"
s_eng[5:8] == s_eng[14:17]	True
"ain" in s_eng	True
"ain" not in s_eng	False
len(s_eng)	43
"1" + "1"	"11"
2 * "1"	"11"
"1" * 2	"11"

שימוש ב-str

ביטוי	ערך
s_eng[:]	"The rain in Spain stays mainly in the plain"
s_eng[2]	"e"
s_eng[:2]	"Th"
s_eng[2:]	"The rain in Spain stays mainly in the plain"
s_eng[2:7]	"e rai"
s_eng[2:7:2]	"eri"
s_eng[5:8] == s_eng[14:17]	True
"ain" in s_eng	True
"ain" not in s_eng	False
len(s_eng)	43
"1" + "1"	"11"
2 * "1"	"11"
"1" * 2	"11"

הפעולה נקראת
slicing (פריסה)

זיהוי טקסט כפול

- טקסט הוא מהצורה WW אם חציו הראשון וחציו השני זהים.

```
Python 3.4.1: testww.py - /Users/yrabani/Dropbox/סדרות/שיעור 2 - פייתון/תשעז/מחשב - למדעי המחשב/Code/testww.py
def is_ww(text):
    """ test if text is of the form ww
        text is a string
        returned value is True / False """
    assert(type(text) == str)
    if len(text)%2 != 0:
        return False
    else:
        for i in range(len(text)//2):
            if text[i] != text[i + len(text)//2]:
                return False
        return True

def prt_ww(text):
    """ create a message stating if text is of the form ww or not
        text is a string
        returned value is one of two strings """
    assert(type(text) == str)
    if is_ww(text):
        message = " "
    else:
        message = " not "
    return "" + text + "" + " is" + message + "of the form WW."

print(prt_ww("Write this once. "))
print(prt_ww("Write this twice. Write this twice. "))
```

Ln: 22 Col: 25

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
```

```
>>> ===== RESTART =====
```

```
>>>
```

```
'Write this once.' is not of the form WW.
```

```
'Write this twice. Write this twice. ' is of the form WW.
```

```
>>>
```

Ln: 8 Col: 4

```
def is_ww(text):
    """ test if text is of the form ww
        text is a string
        returned value is True / False """
    assert(type(text) == str)
    if len(text)%2 != 0:
        return False
    else:
        for i in range(len(text)//2):
            if text[i] != text[i + len(text)//2]:
                return False
        return True

def prt_ww(text):
    """ create a message stating if text is of the form ww or not
        text is a string
        returned value is one of two strings """
    assert(type(text) == str)
    if is_ww(text):
        message = " "
    else:
        message = " not "
    return "" + text + "" + " is" + message + "of the form WW."

print(prt_ww("Write this once. "))
print(prt_ww("Write this twice. Write this twice. "))
```

Ln: 22 Col: 25

זיהוי טקסט כפול

- עוד יותר פשוט:

Python 3.4.1: simpletestww.py - /Users/yrabani/Dropbox/סדרות - שיעור 2/תשעז/פייתון/הוראה/מבוא למדעי המחשב - Code/simplete...

```
def simple_is_ww(text):
    """ test if text is of the form ww
        text is a string
        returned value is True / False """
    assert(type(text) == str)
    if text[:len(text)//2] == text[len(text)//2:]:
        return True
    else:
        return False
```

Ln: 10	Col: 0
--------	--------

הטיפוס tuple

- הטיפוס tuple משמש להחזיק רשימה קבועה של ערכים.

- דוגמאות:

(42, 17, 80)

רשימה של קבועי int -

()

רשימה ריקה -

3 * (0,)

רשימה של ערכים זהים -

tuple(range(3))

ייצור tuple מסדרה אחרת -

tuple('concerto')

(1, 2) + (3,)

שרשור רשימות -

- בדומה ל-str, אפשר להשתמש ב-in, len, וב-[:] כדי להתייחס לאבר או תת-סדרה.

הטיפוס tuple

- הטיפוס tuple משמש להחזיק רשימה קבועה של ערכים.

- דוגמאות:

(42, 17, 80)

רשימה של קבועי int -

()

רשימה ריקה -

3 * (0,) == (0, 0, 0)

רשימה של ערכים זהים -

tuple(range(3))

ייצור tuple מסדרה אחרת -

tuple('concerto')

(1, 2) + (3,)

שרשור רשימות -

- בדומה ל-str, אפשר להשתמש ב-in, len, וב-[:] כדי להתייחס לאבר או תת-סדרה.

הטיפוס tuple

- הטיפוס tuple משמש להחזיק רשימה קבועה של ערכים.

- דוגמאות:

(42, 17, 80)

רשימה של קבועי int -

()

רשימה ריקה -

3 * (0,) == (0, 0, 0)

רשימה של ערכים זהים -

tuple(range(3)) == (0, 1, 2)

ייצור tuple מסדרה אחרת -

tuple('concerto')

(1, 2) + (3,)

שרשור רשימות -

- בדומה ל-str, אפשר להשתמש ב-in, len, וב-[:] כדי להתייחס לאבר או תת-סדרה.

הטיפוס tuple

- הטיפוס tuple משמש להחזיק רשימה קבועה של ערכים.

- דוגמאות:

(42, 17, 80)		רשימה של קבועי int -
()		רשימה ריקה -
3 * (0,)	== (0, 0, 0)	רשימה של ערכים זהים -
tuple(range(3))	== (0, 1, 2)	ייצור tuple מסדרה אחרת -
tuple('concerto')	== ('c', 'o', 'n', 'c', 'e', 'r', 't', 'o')	
(1, 2) + (3,)		שרשור רשימות -

- בדומה ל-str, אפשר להשתמש ב-in, not in, len, וב-[: :] כדי להתייחס לאבר או תת-סדרה.

הטיפוס tuple

- הטיפוס tuple משמש להחזיק רשימה קבועה של ערכים.

- דוגמאות:

(42, 17, 80)		רשימה של קבועי int -
()		רשימה ריקה -
3 * (0,)	== (0, 0, 0)	רשימה של ערכים זהים -
tuple(range(3))	== (0, 1, 2)	ייצור tuple מסדרה אחרת -
tuple('concerto')	== ('c', 'o', 'n', 'c', 'e', 'r', 't', 'o')	
(1, 2) + (3,)	== (1, 2, 3)	שרשור רשימות -

- בדומה ל-str, אפשר להשתמש ב-in, not in, len, וב-[:] כדי להתייחס לאבר או תת-סדרה.

דוגמה

- נקודה במפה מיוצגת על ידי קו רוחב (צפון או דרום, מעלות, דקות, שניות) וקו אורך (מזרח או מערב, מעלות, דקות, שניות). - ייצוג DMS
- במערכות GPS מייצגים את קווי הרוחב והאורך על ידי זוג מספרים. הסימן מייצג את הכיוון (צפון ומזרח - סימן חיובי) והערך הוא מעלות וחלקי מעלה. - ייצוג DD
- במעלה יש 60 דקות ובדקה 60 שניות.
- אנחנו נכתוב תוכנית שמתרגמת ייצוג DMS לייצוג DD.

קוד להמרת DMS ל-DD

Python 3.4.1: DMStoDD.py - /Users/yrabani/Documents/3 פייתון/תשעה/שיעור/Code/DMStoDD.py

```
DMS_location = (("N", (31, 46, 37)), ("E", (35, 11, 54)))

latitude = DMS_location[0]
longitude = DMS_location[1]

lat_coord = latitude[1]
long_coord = longitude[1]

lat_val = lat_coord[0] + lat_coord[1]/60 + lat_coord[2]/3600
if latitude[0] == "S":
    lat_val = -lat_val
long_val = long_coord[0] + long_coord[1]/60 + long_coord[2]/3600
if longitude[0] == "W":
    long_val = -long_val

DD_location = (lat_val, long_val)

print(DD_location)
```

Ln: 1 Col: 0

(31.776944444444442, 35.19833333333333)

99 בקבוקי בירה

Python 3.4.1: beer.py - /Users/ywabani/Documents/3 פייתון/תשעה/שיעור/Code/beer.py

```
# print the song:
# Nine bottles of beer on the wall,
# Nine bottles of beer;
# If one of those bottles should
#     happen to fall,
# Eight bottles of beer on the wall.
# ...
#
# The original song starts with Ninety nine bottles ...
#
numbers = ('No', 'One', 'Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine')
bottles = " bottles of beer"
wall = " on the wall"
fall = "If one of those bottles should\n    happen to fall,\n"

for i in range(len(numbers)-1, 0, -1):
    print(numbers[i] + bottles + wall + ",")
    print(numbers[i] + bottles + ";")
    print(fall + numbers[i-1] + bottles + wall + ".\n")
```

סדרות קבועות ומשתנות

- בפייתון מבחינים בין:
 - אובייקט ניתן לשינוי (mutable)
 - אובייקט שערכו קבוע (immutable)
- בפרט range, str, tuple הם immutable.
- אנחנו נראה עוד מעט טיפוס של סדרה שהוא mutable.
- עוד על הנושא הזה - בהמשך השיעור

הטיפוס list

- הטיפוס הזה דומה מאוד ל-tuple, למעט העובדה שאובייקט מטיפוס list ניתן לשינוי (mutable).

- דוגמאות:

רשימה ריקה

```
[]
```

רשימה של שישה קבועים

```
[0, 1, 3, 6, 10, 15]
```

ייצור list מסדרה אחרת

```
list((1, 4, 9, 16))
```

לחילופין (נבין מאיפה זה בא בהמשך)

```
[i**2 for i in range(1,5)]
```

השמה למשתנה

```
looknsay = [1, 11, 21, 1211, 111221, 312211]
```

איפוס רשימה

```
4 * [0]
```

שרשור

```
["a", "b"] + ["b", "a"]
```

הוספת אבר בסוף הרשימה

```
looknsay.append(13112221)
```

- גם כאן אפשר להשתמש ב-len, in, not in, וב-[:]

הטיפוס list

- הטיפוס הזה דומה מאוד ל-tuple, למעט העובדה שאובייקט מטיפוס list ניתן לשינוי (mutable).

- דוגמאות:

[]	רשימה ריקה
[0, 1, 3, 6, 10, 15]	רשימה של שישה קבועים
list((1, 4, 9, 16))	ייצור list מסדרה אחרת
[i**2 for i in range(1,5)]	לחילופין (נבין מאיפה זה בא בהמשך)
looknsay = [1, 11, 21, 1211, 111221, 312211]	השמה למשתנה
4 * [0] == [0, 0, 0, 0]	איפוס רשימה
["a", "b"] + ["b", "a"]	שרשור
looknsay.append(13112221)	הוספת אבר בסוף הרשימה

- גם כאן אפשר להשתמש ב-len, in, not in, וב-[: :]

הטיפוס list

- הטיפוס הזה דומה מאוד ל-tuple, למעט העובדה שאובייקט מטיפוס list ניתן לשינוי (mutable).

- דוגמאות:

רשימה ריקה

```
[ ]
```

רשימה של שישה קבועים

```
[0, 1, 3, 6, 10, 15]
```

ייצור list מסדרה אחרת

```
list((1, 4, 9, 16))
```

לחילופין (נבין מאיפה זה בא בהמשך)

```
[i**2 for i in range(1,5)]
```

השמה למשתנה

```
looknsay = [1, 11, 21, 1211, 111221, 312211]
```

איפוס רשימה

```
4 * [0] == [0, 0, 0, 0]
```

שרשור

```
["a", "b"] + ["b", "a"] == ["a", "b", "b", "a"]
```

הוספת אבר בסוף הרשימה

```
looknsay.append(13112221)
```

- גם כאן אפשר להשתמש ב-len, in, not in, וב-[:]

הנפה של ארטוסתנס

(Sieve of Eratosthenes)

Python 3.4.1: Eratosthenes.py - /Users/ygrabani/Documents/3 פיית'ון/תשעה/שיעור 3/Code/Eratosthenes...

```
# find and print the prime numbers up to N, using
# the Sieve of Eratosthenes

N = 100

sieve = list(range(N+1))
# alternatively, we can write: sieve = [i for i in range(N+1)]

for i in range(2, len(sieve)):      # find primes starting with 2
    if sieve[i] != 0:               # a number that wasn't erased is a prime
        print(i)
        for j in range(2*i, len(sieve), i):    # erase all this prime's multiples
            sieve[j] = 0

print(sieve)
```

Ln: 12 Col: 78

הנפה של ארטוסתנס (Sieve of Eratosthenes)

Python 3.4.1: Eratosthenes.py - /Users/ygrabani/Documents/3 שיעור/תשעה/פיית'ון/מבוא למדעי המחשב - הוראה/Code/Eratosthenes...

```
# find and print the prime numbers up to N, using
# the Sieve of Eratosthenes

N = 100

sieve = list(range(N+1))
# alternatively, we can write: sieve = [i for i in range(N+1)]

for i in range(2, len(sieve)):      # find primes starting with 2
    if sieve[i] != 0:              # a number that wasn't erased is a prime
        print(i)
        for j in range(2*i, len(sieve), i):  # erase all this prime's multiples
            sieve[j] = 0

print(sieve)
```

זו פעולה שלא ניתן היה לעשות לוא היינו
מאתחלים על ידי: `sieve = tuple(range(N+1))`
כי tuple הוא immutable

תחום הגדרה של משתנים

תחום הגדרה של משתנים

- המשתנים בתוך פונקציה לא מוכרים מחוצה לה. אין אפשרות לגשת אליהם:

```
def a(x):  
    y = x+2  
    return y
```

```
a(1)  
print(x)  
print(y)
```

תחום הגדרה של משתנים

- המשתנים בתוך פונקציה לא מוכרים מחוצה לה. אין אפשרות לגשת אליהם:

```
def a(x):  
    y = x+2  
    return y
```

```
a(1)
```

```
print(x)  
print(y)
```

זו שגיאה!

תחום הגדרה של משתנים

- המשתנים בתוך פונקציה לא מוכרים מחוצה לה. אין אפשרות לגשת אליהם:

```
def a(x):  
    y = x+2  
    return y
```

זה מאפשר לשנות את המימוש של הפונקציה בלי לשנות את הקוד המשתמש בה.

a(1)

```
print(x)  
print(y)
```

זו שגיאה!

זה גם מונע השפעה שלא דרך הממשק על פעולת הפונקציה.

תחום הגדרה של משתנים

- לעומת זאת, הפונקציה יכולה לגשת למשתנים שהוגדרו בבלוק המכיל אותה:

```
y = 2
def a():
    print(y)
y = 3
a()
```

תחום הגדרה של משתנים

- לעומת זאת, הפונקציה יכולה לגשת למשתנים שהוגדרו בבלוק המכיל אותה:

```
y = 2
```

```
def a():
```

```
    print(y)
```

```
y = 3
```

```
a()
```

מה שיודפס כאן הוא 3



תחום הגדרה של משתנים

- לעומת זאת, הפונקציה יכולה לגשת למשתנים שהוגדרו בבלוק המכיל אותה:

```
y = 2
```

```
def a():
```

```
    print(y)
```

```
y = 3
```

```
a()
```

רצוי להימנע מקוד כזה.

הוא מאוד מבלבל.

עדיף להעביר ערכים חיצוניים נדרשים כפרמטרים

יוצאי דופן - קבועים אוניברסליים

מה שיודפס כאן הוא 3

תחום הגדרה של משתנים

- אם מבצעים בתוך פונקציה פקודת השמה למשתנה, אז פייתון מייצר משתנה פנימי של הפונקציה, גם אם יש משתנה חיצוני בעל אותו שם:

```
x = 1  
y = 2  
def f(x):  
    x = x+1  
    y = x+1
```

```
z = f(1)  
print(x, y)
```

תחום הגדרה של משתנים

- אם מבצעים בתוך פונקציה פקודת השמה למשתנה, אז פייתון מייצר משתנה פנימי של הפונקציה, גם אם יש משתנה חיצוני בעל אותו שם:

```
x = 1  
y = 2  
def f(x):  
    x = x+1  
    y = x+1
```

```
z = f(1)
```

```
print(x, y)
```

מה שיודפס כאן הוא 1,2

תחום הגדרה של משתנים

- אם מבצעים בתוך פונקציה פקודת השמה למשתנה, אז פייתון מייצר משתנה פנימי של הפונקציה, גם אם יש משתנה חיצוני בעל אותו שם:

```
x = 1  
y = 2  
def f(x):  
    x = x+1  
    y = x+1
```

זה חוסם השפעה של הפונקציה על הקוד הקורא לה שלא דרך הממשק.

```
z = f(1)  
print(x, y)
```

מה שיודפס כאן הוא 1,2

השפעות לוואי

- אפשר לעקוף את החסימה של פייתון:

```
x = 1
def a():
    global x
    x = x+1
a()
a()
print(x)
```

השפעות לוואי

- אפשר לעקוף את החסימה של פייתון:

```
x = 1  
def a():  
    global x  
    x = x+1
```

```
a()
```

```
a()
```

```
print(x)
```

מה שיודפס כאן הוא 3

השפעות לוואי

- אפשר לעקוף את החסימה של פייתון:

```
x = 1  
def a():  
    global x  
    x = x+1
```

קוד גרוע!

יש להימנע משימוש ב-global

```
a()
```

```
a()
```

```
print(x)
```

מה שיודפס כאן הוא 3

קינון

```
x = 5
y = 2
def f1(x):
    y = 1
    def f2(y):
        return y**2
    print(y)
    return f2(f2(x))

print(f1(y), x)

def f3(x):
    print(x)
```

משתנים ואובייקטים

מראי מקום (references)

- משתנה אינו מחזיק ערך, אלא כתובת בזיכרון שבה רשום הערך (המשתנה "מצביע" על מיקום הערך).
- הכתובת שבה מוחזק הערך לרוב לא מעניינת אותנו.

```
x = 3  
y = "hello"
```

מראי מקום (references)

- משתנה אינו מחזיק ערך, אלא כתובת בזיכרון שבה רשום הערך (המשתנה "מצביע" על מיקום הערך).
- הכתובת שבה מוחזק הערך לרוב לא מעניינת אותנו.

```
x = 3  
y = "hello"
```

x: 27191

y: 22130

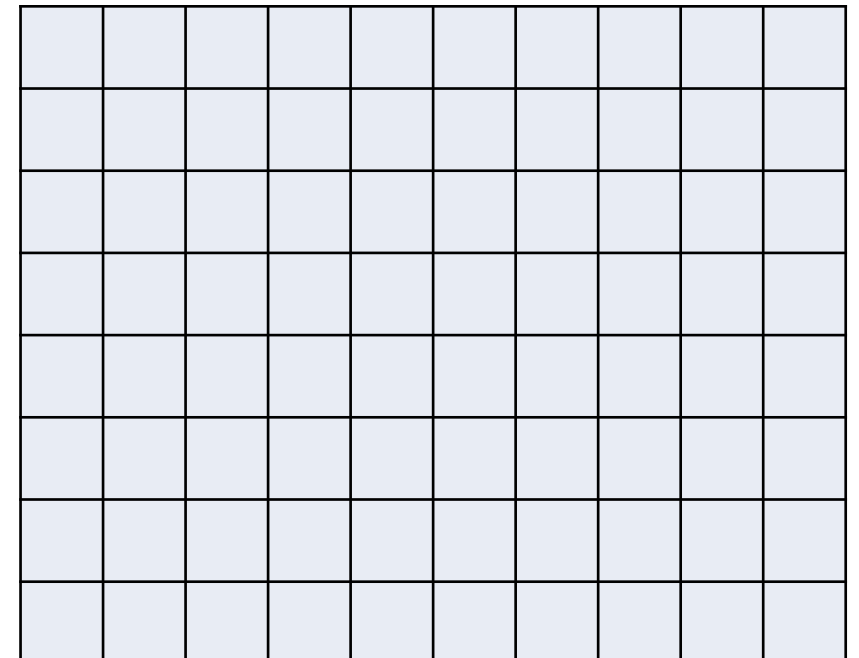
מראי מקום (references)

- משתנה אינו מחזיק ערך, אלא כתובת בזיכרון שבה רשום הערך (המשתנה "מצביע" על מיקום הערך).
- הכתובת שבה מוחזק הערך לרוב לא מעניינת אותנו.

```
x = 3  
y = "hello"
```

x: 27191

y: 22130



מראי מקום (references)

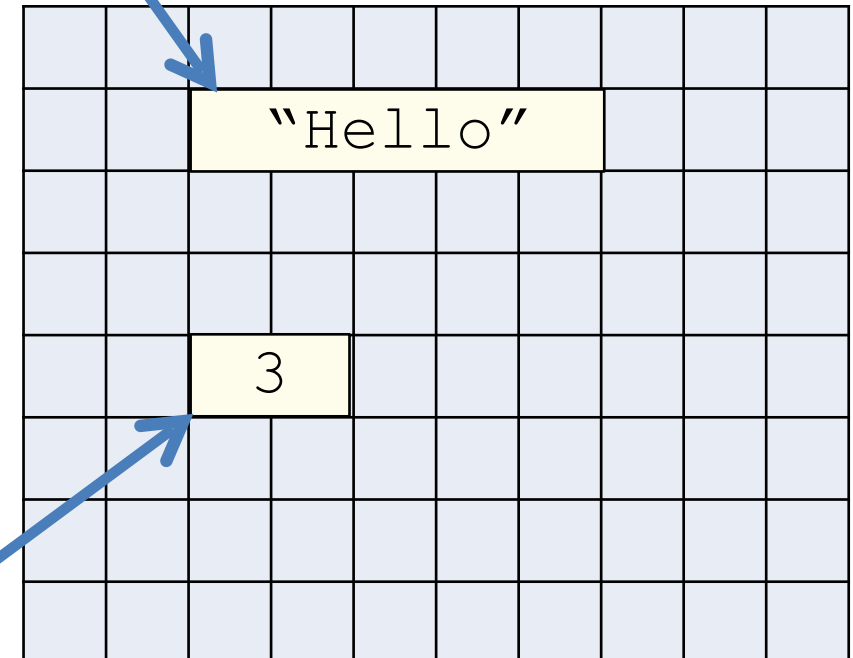
- משתנה אינו מחזיק ערך, אלא כתובת בזיכרון שבה רשום הערך (המשתנה "מצביע" על מיקום הערך).
- הכתובת שבה מוחזק הערך לרוב לא מעניינת אותנו.

```
x = 3  
y = "hello"
```

x: 27191

y: 22130

תא זיכרון 22130



תא זיכרון 27191

פקודת השמה

- פקודת השמה מעתיקה את **הכתובת** (לא את הערך)

```
x = "hello"
```

```
y = x
```

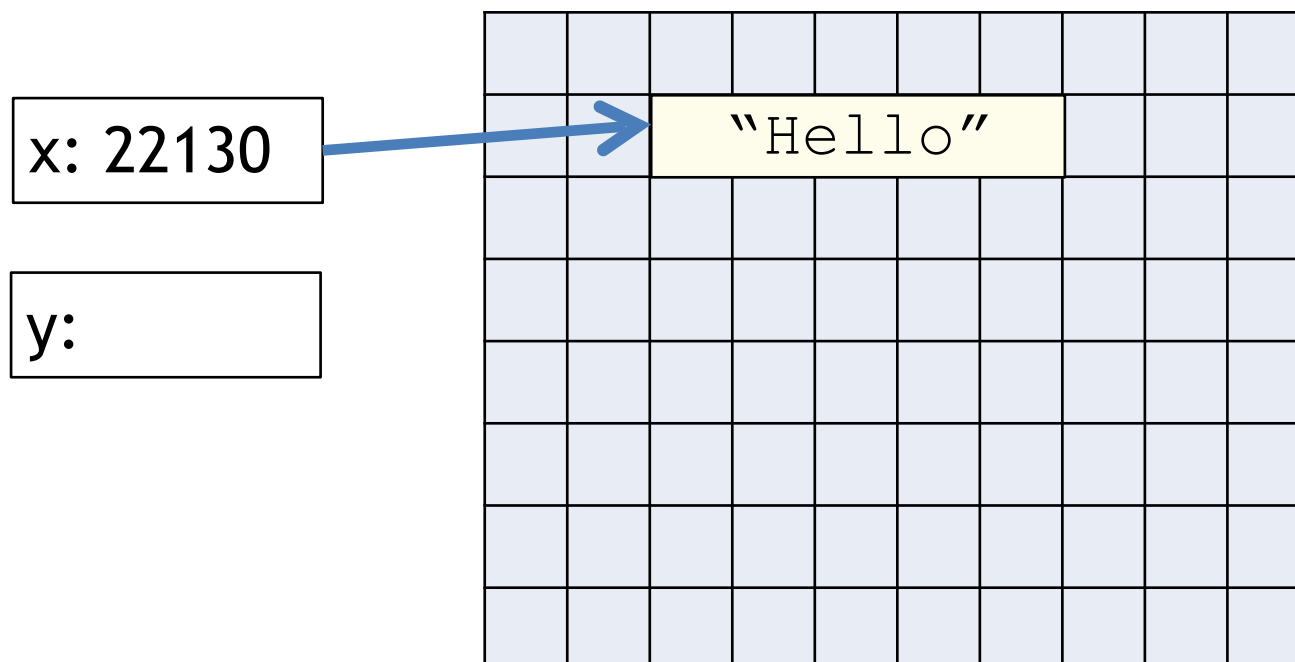
```
print(id(x))
```

```
Print(id(y))
```

הפונקציה id מחזירה את "מספר הזהות" של האובייקט שהמשתנה מצביע עליו (תלוי מימוש, לרוב זו הכתובת)

מובטח שה-id של אובייקט נותר קבוע כל עוד הוא "בחיים".

אותו id יכול לשמש אובייקטים שונים אם אין חפיפה בין קטעי הזמן שבהם הם בחיים



פקודת השמה

- פקודת השמה מעתיקה את **הכתובת** (לא את הערך)

```
x = "hello"
```

```
y = x
```

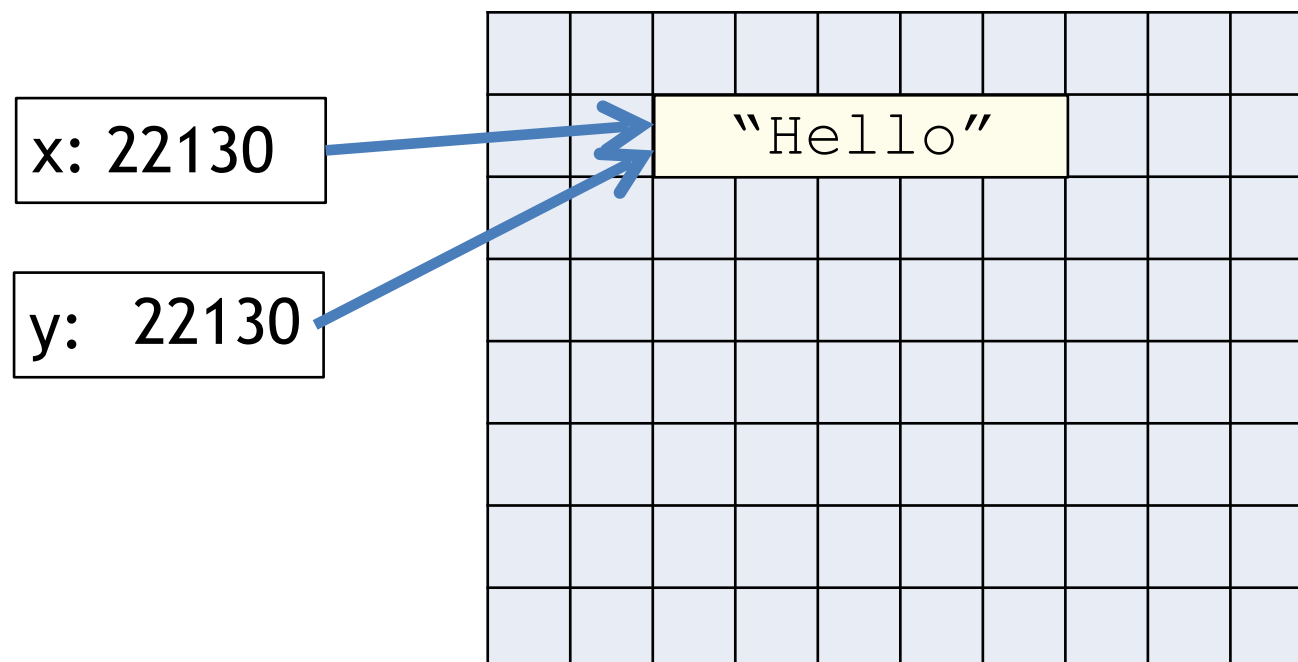
```
print(id(x))
```

```
Print(id(y))
```

הפונקציה id מחזירה את "מספר הזהות" של האובייקט שהמשתנה מצביע עליו (תלוי מימוש, לרוב זו הכתובת)

מובטח שה-id של אובייקט נותר קבוע כל עוד הוא "בחיים".

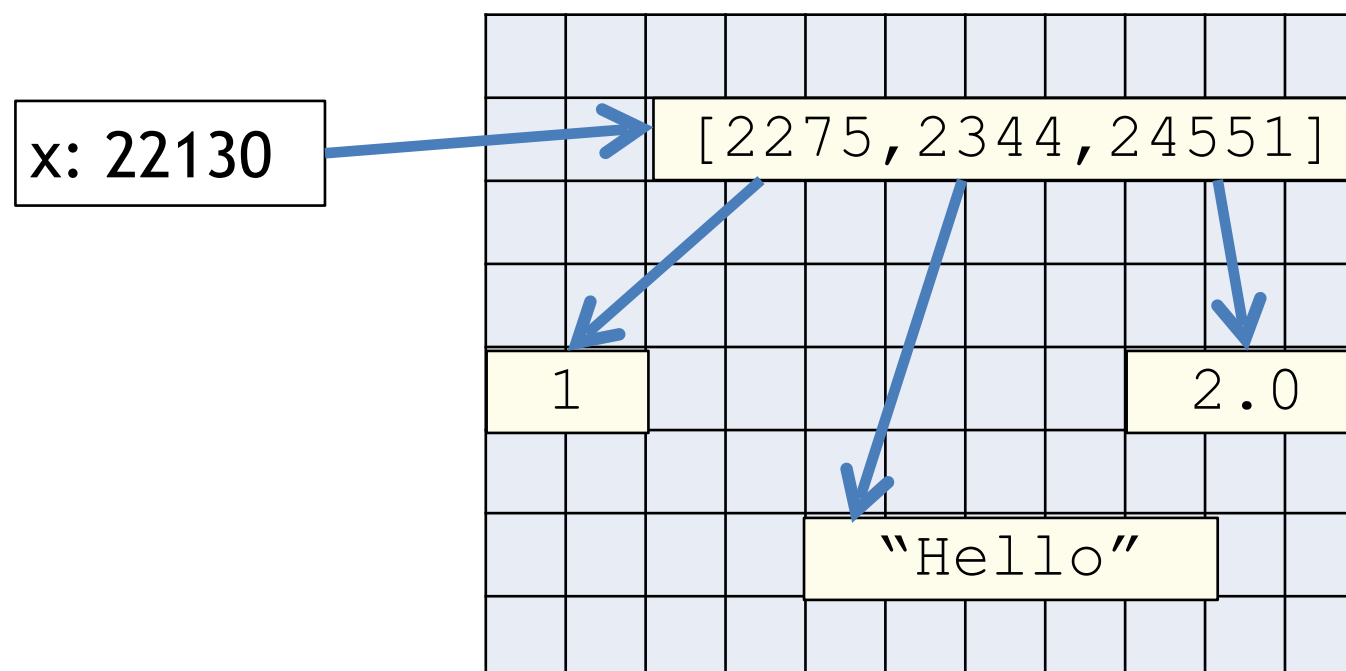
אותו id יכול לשמש אובייקטים שונים אם אין חפיפה בין קטעי הזמן שבהם הם בחיים



מיכלים

- במיכל (למשל, אובייקט מטיפוס list), כל אבר הוא מראה מקום (reference) של הערך של האבר
- משתנה list הוא מראה מקום של אובייקט list, שהוא בעצמו סדרה של מראי מקום:

```
x = [1, "hello", 2.0]
```



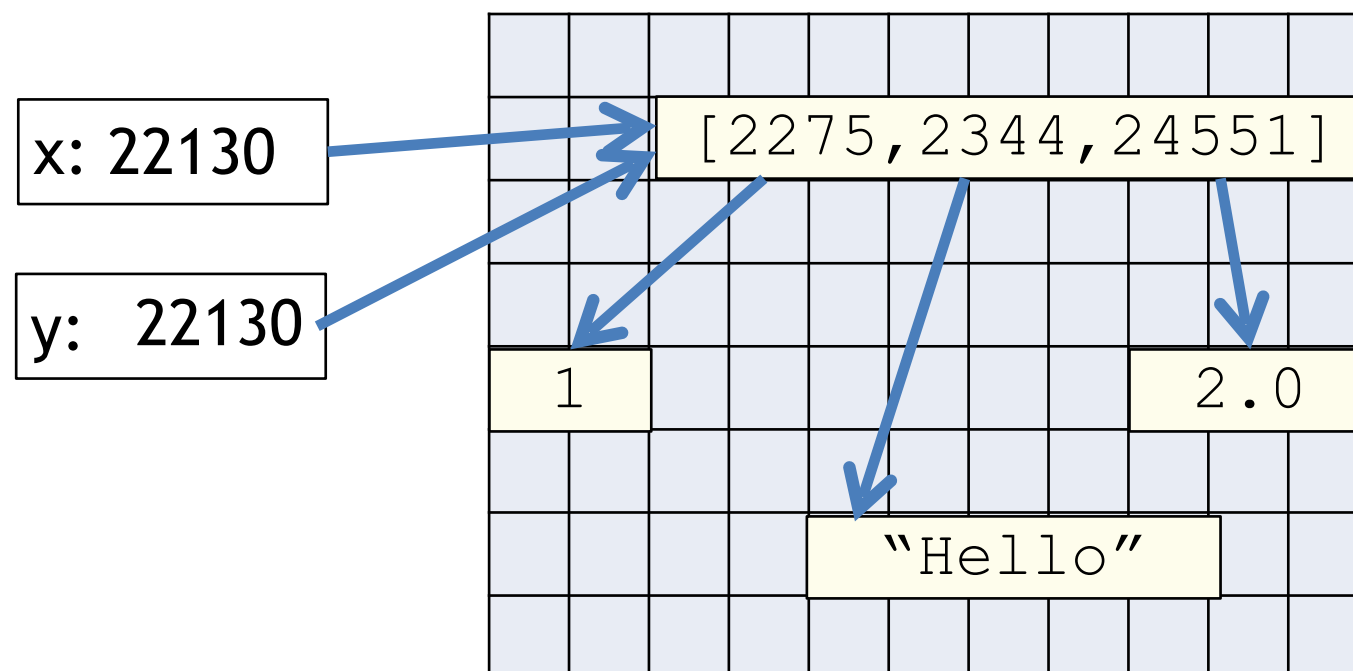
השלכות

```
x = [1, "hello", 2.0]
```

```
y = x
```

```
y[0] = 53
```

```
print(x[0])
```



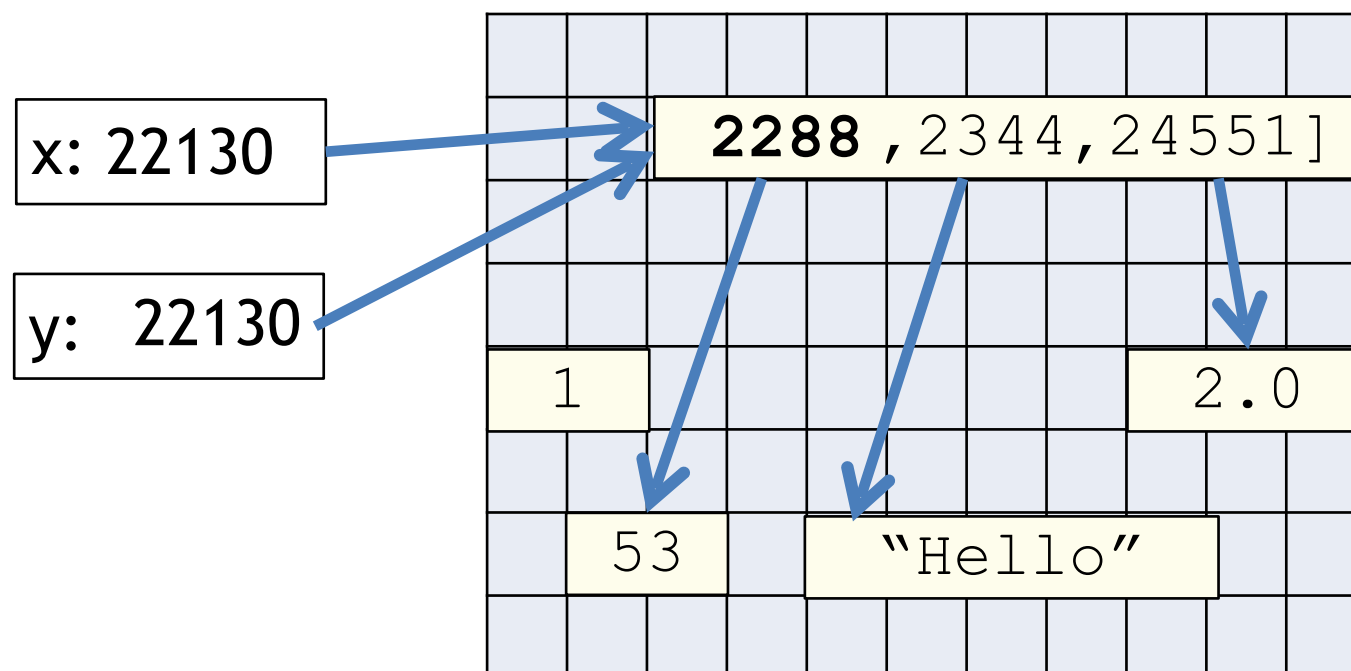
השלכות

```
x = [1, "hello", 2.0]
```

```
y = x
```

```
y[0] = 53
```

```
print(x[0])
```



השלכות

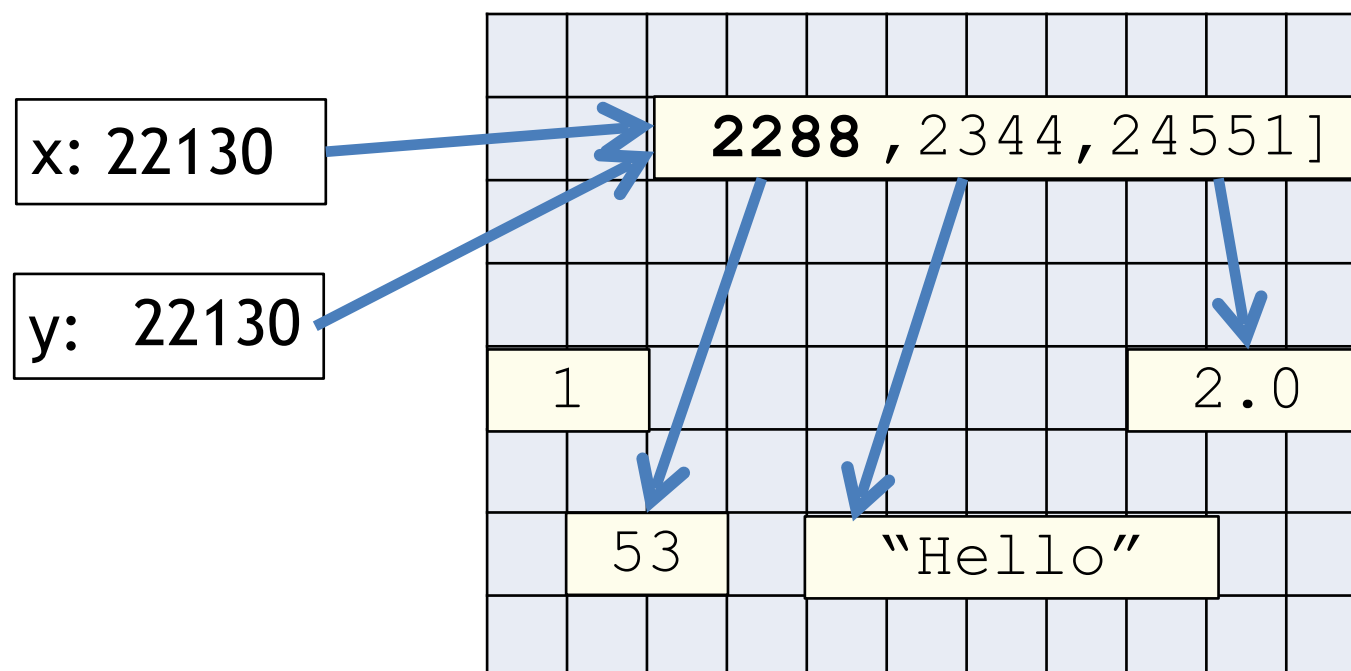
```
x = [1, "hello", 2.0]
```

```
y = x
```

```
y[0] = 53
```

```
print(x[0])
```

יודפס 53



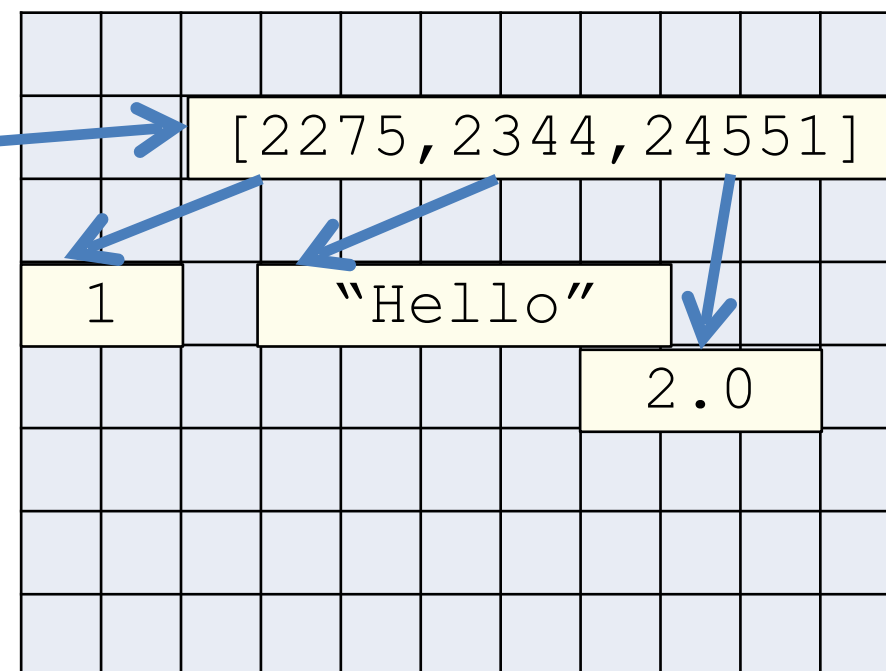
ולעומת זאת ...

```
x = [1, "hello", 2.0]  
y = x[:]
```

פריסה (slicing) יוצרת עותק חדש של הרשימה

זה עותק "רדוד" (shallow copy):
רק מראי המקום ברשימה מועתקים

x: 22130



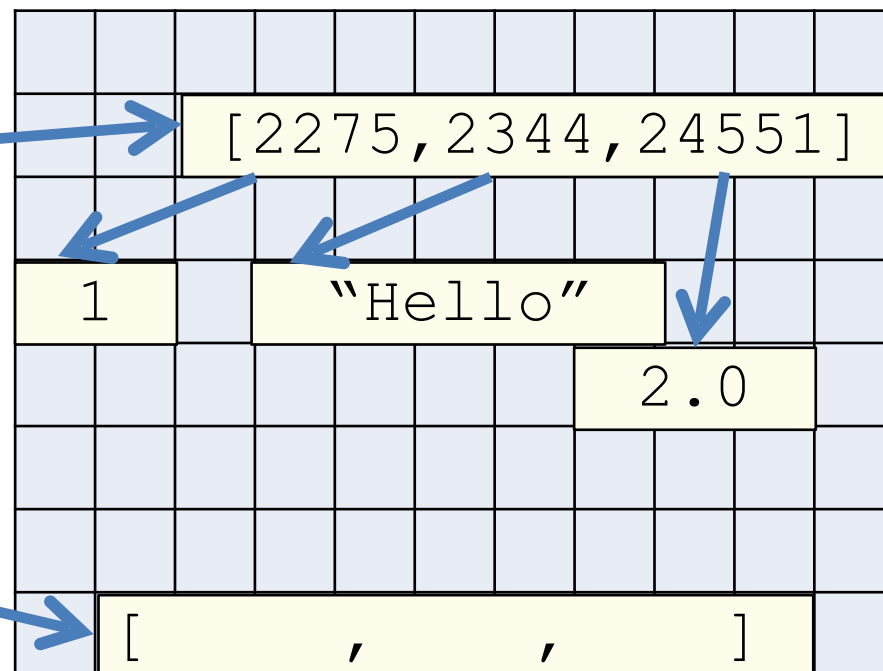
ולעומת זאת ...

```
x = [1, "hello", 2.0]
y = x[:]
```

פריסה (slicing) יוצרת עותק חדש של הרשימה

זה עותק "רדוד" (shallow copy):
רק מראי המקום ברשימה מועתקים

x: 22130



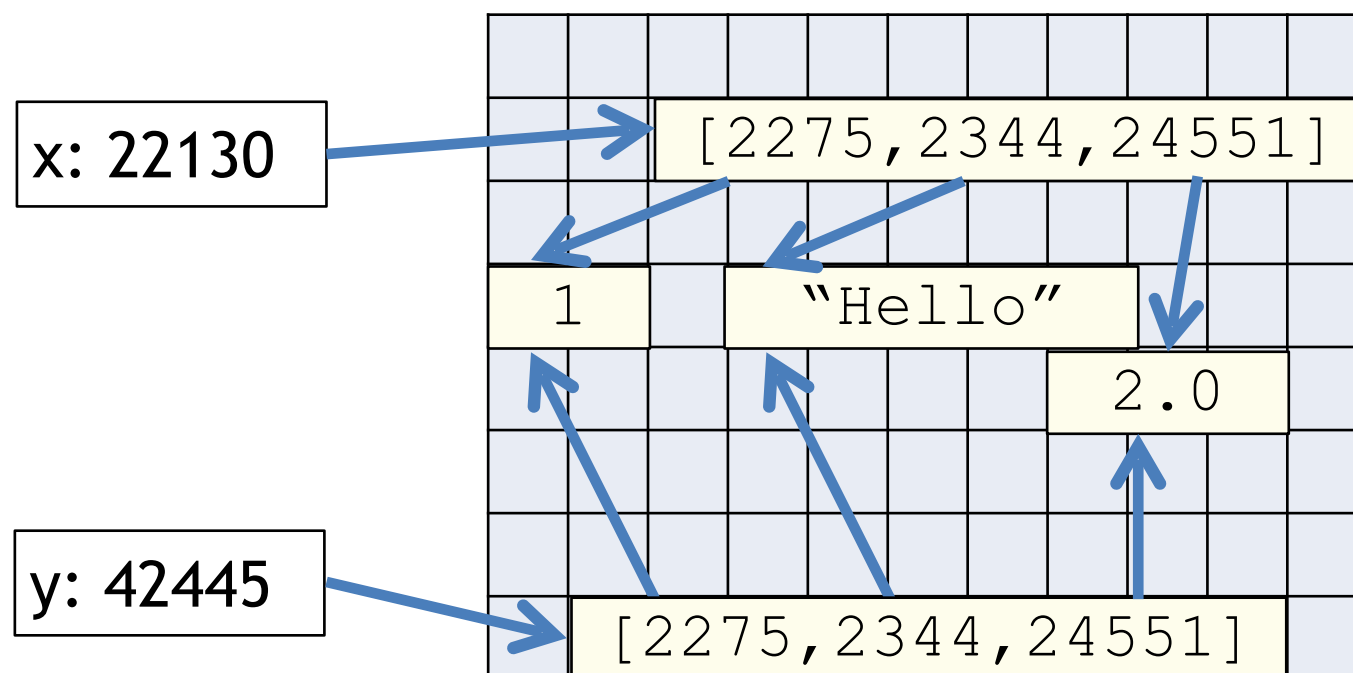
y: 42445

ולעומת זאת...

```
x = [1, "hello", 2.0]
y = x[:] ←
```

פריסה (slicing) יוצרת עותק חדש של הרשימה

זה עותק "רדוד" (shallow copy):
רק מראי המקום ברשימה מועתקים



ולעומת זאת...

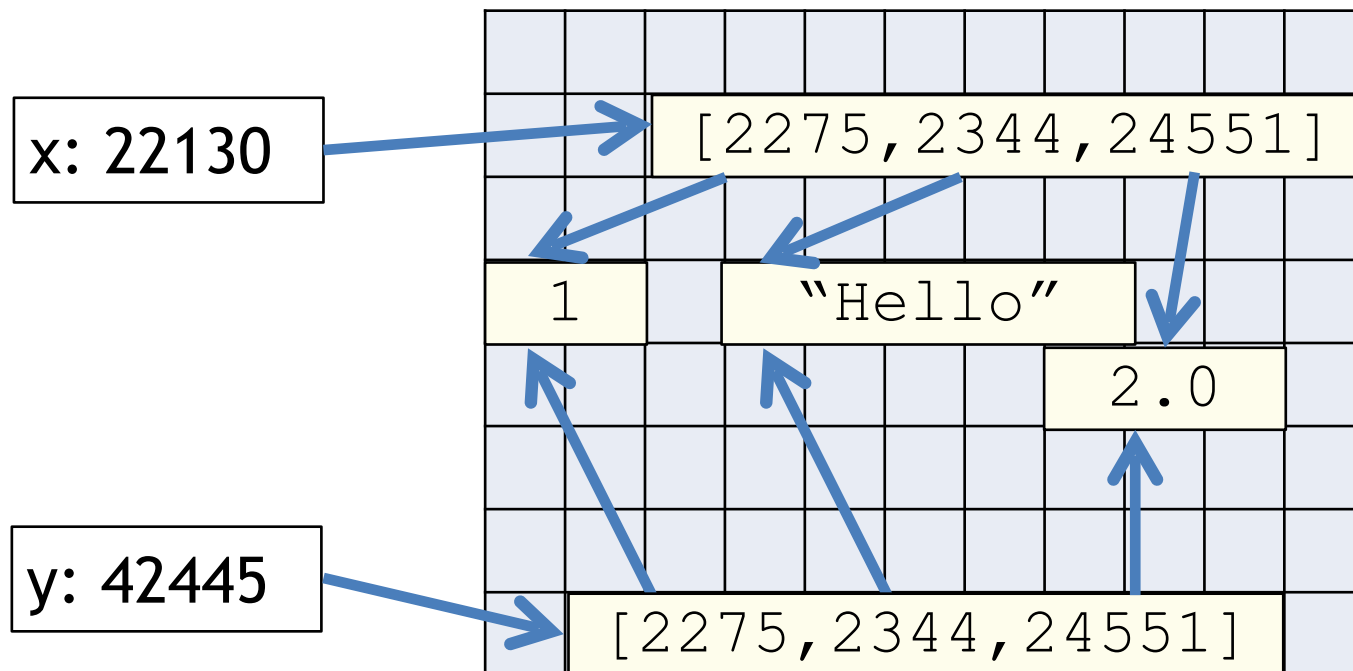
```
x = [1, "hello", 2.0]
```

```
y = x[:, ]
```

```
y[0] = 53
```

פריסה (slicing) יוצרת עותק חדש של הרשימה

זה עותק "רדוד" (shallow copy):
רק מראי המקום ברשימה מועתקים



ולעומת זאת...

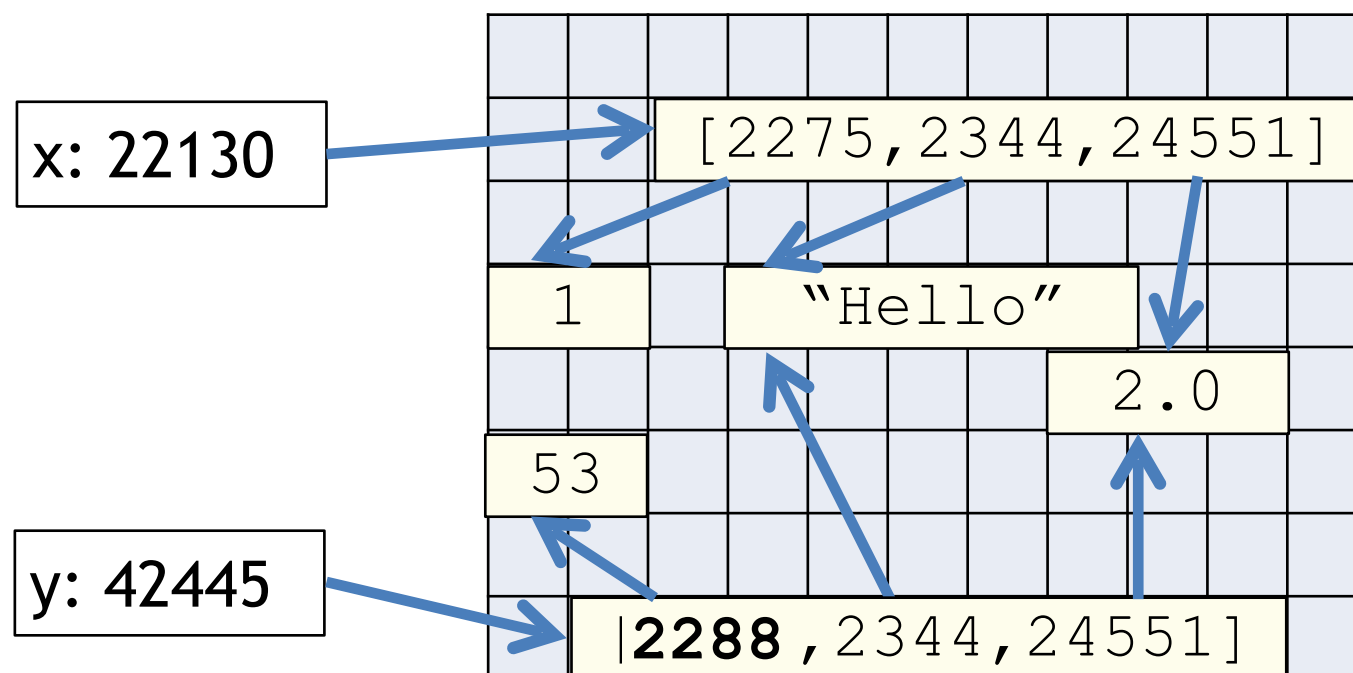
```
x = [1, "hello", 2.0]
```

$$\mathbf{y} = \mathbf{x}[:,]$$

```
y[0] = 53
```

פריסה (slicing) יוצרת עותק חדש של הרשימה

זה עותק "רדוד" (shallow copy):
רק מראי המקום ברשימה מועתקים



ולעומת זאת ...

```
x = [1, "hello", 2.0]
```

```
y = x[:]
```

```
y[0] = 53
```

```
print(x[0])
```

פריסה (slicing) יוצרת עותק חדש של הרשימה

זה עותק "רדוד" (shallow copy):
רק מראי המקום ברשימה מועתקים

x: 22130

[2275, 2344, 24551]

1

"Hello"

2.0

53

y: 42445

[2288, 2344, 24551]

ולעומת זאת ...

```
x = [1, "hello", 2.0]
```

```
y = x[:]
```

```
y[0] = 53
```

```
print(x[0])
```

יודפס 1

פריסה (slicing) יוצרת עותק חדש של הרשימה

זה עותק "רדוד" (shallow copy):
רק מראי המקום ברשימה מועתקים

x: 22130

[2275, 2344, 24551]

1

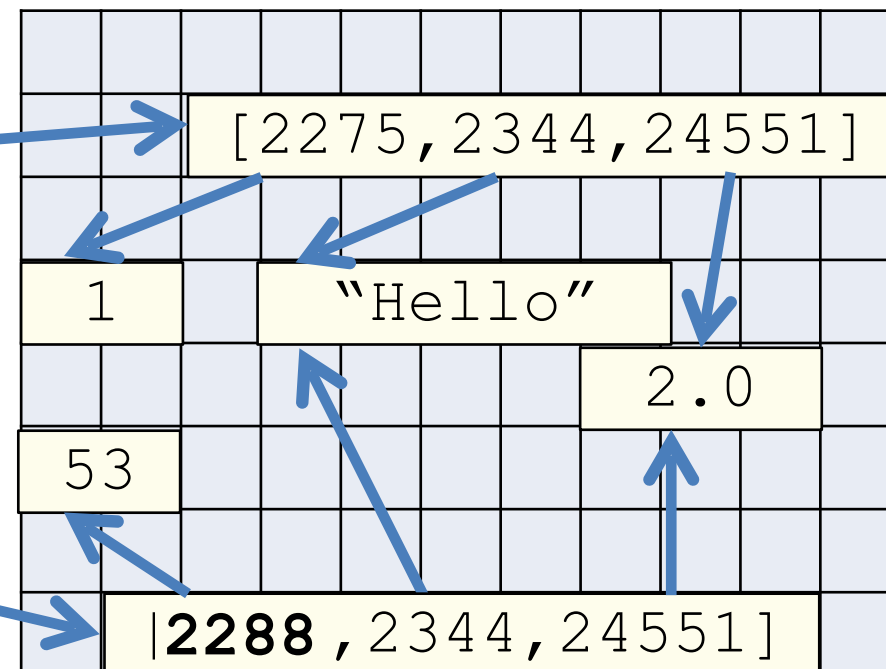
"Hello"

2.0

53

y: 42445

[2288, 2344, 24551]



השוואת רשימות

```
x = [1, "hello", 2.0]
```

השוואת רשימות

```
x = [1, "hello", 2.0]  
y = x[:]
```

השוואת רשימות

```
x = [1, "hello", 2.0]  
y = x[:]  
z = x
```

השוואת רשימות

```
x = [1, "hello", 2.0]
y = x[:]
z = x

print(x is y)    #False
```

השוואת רשימות

```
x = [1, "hello", 2.0]
y = x[:]
z = x

print(x is y)      #False
print(x == y)     #True
```

השוואת רשימות

```
x = [1, "hello", 2.0]
y = x[:]
z = x

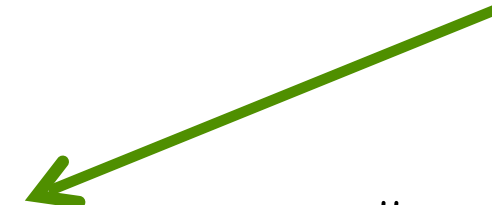
print(x is y)      #False
print(x == y)     #True
print(z is x)      #True
```

השוואת רשימות

```
x = [1, "hello", 2.0]
y = x[:]
z = x
```

```
print(x is y)      #False
print(x == y)     #True
print(z is x)     #True
```

אותה רשימה? (אותו
מקום בזיכרון?)



השוואת רשימות

```
x = [1, "hello", 2.0]
y = x[:]
z = x
```

אותה רשימה? (אותו
מקום בזיכרון?)

```
print(x is y)    #False
print(x == y)    #True
print(z is x)    #True
```

אותו תוכן?

עוד פעולות

```
x = [1, 2, 3]
y = x + [4, 5, 6]

print(x is y)    #False
```

אופרטור השרשור +
יוצר list חדש

```
x = [1, 2, 3]
x.extend([4, 5, 6])
x.append(7)
```

append()-ו extend()
משנים list קיים

```
x = [1, 2, 3]
y = copy.copy(x)
z = list(x)

print(y is x)    #False
print(z is x)    #False
```

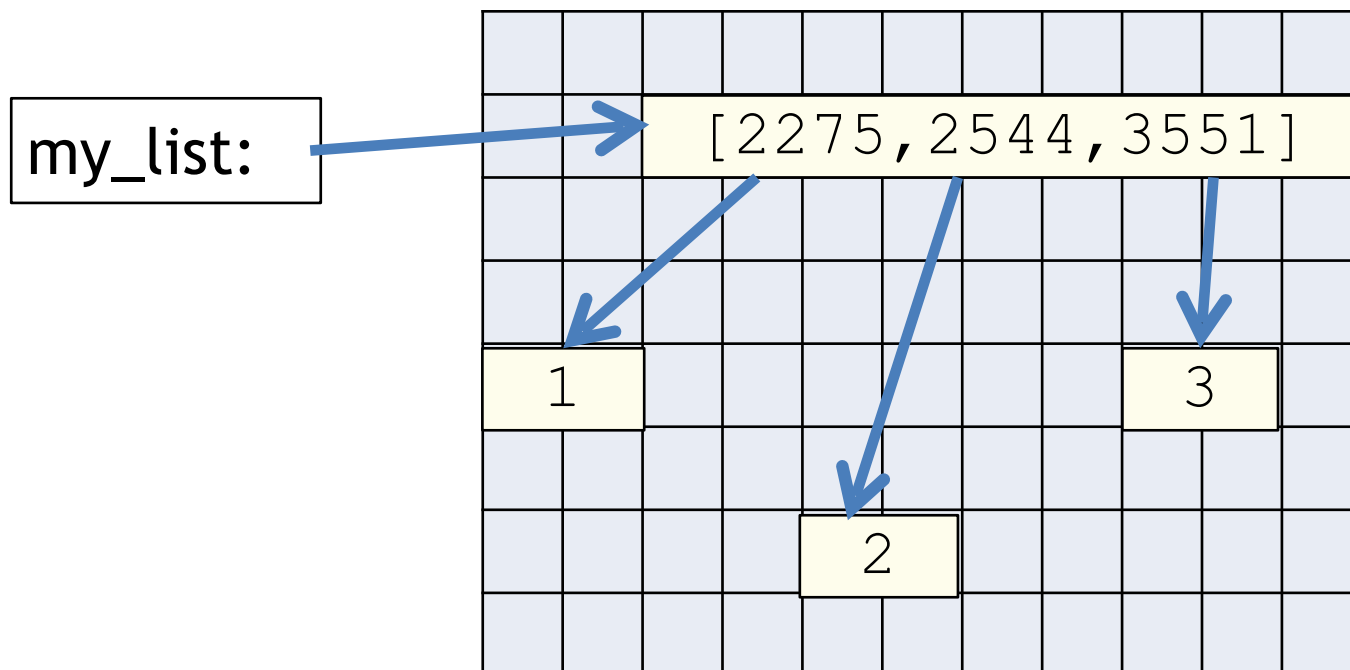
שיטות נוספות ליצור
עותק חדש של list

רשימות ופונקציות

- אפשר להעביר לפונקציה רשימה כפרמטר:

```
def set_zero(lst, i):  
    lst[i] = 0
```

```
my_list = [1, 2, 3]
set_zero(my_list, 1)
print(my_list)
```

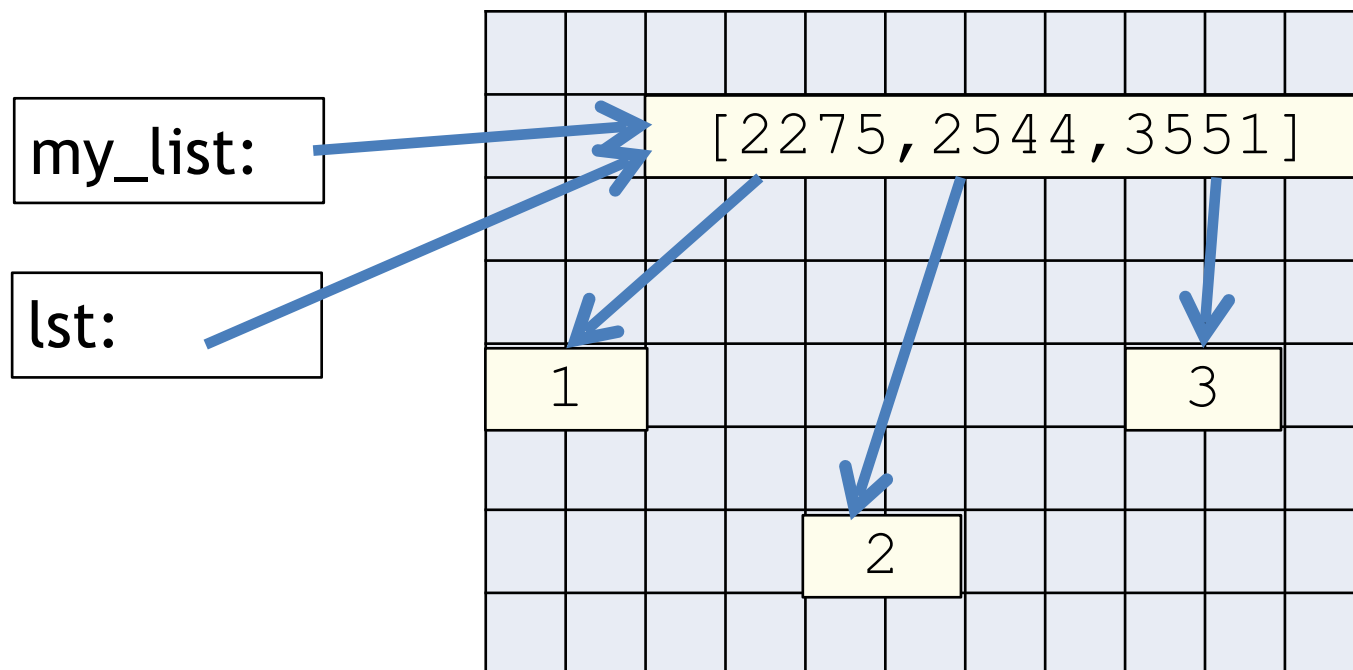


רשימות ופונקציות

- אפשר להעביר לפונקציה רשימה כפרמטר:

```
def set_zero(lst, i):  
    lst[i] = 0
```

```
my_list = [1, 2, 3]  
set_zero(my_list, 1)  
print(my_list)
```

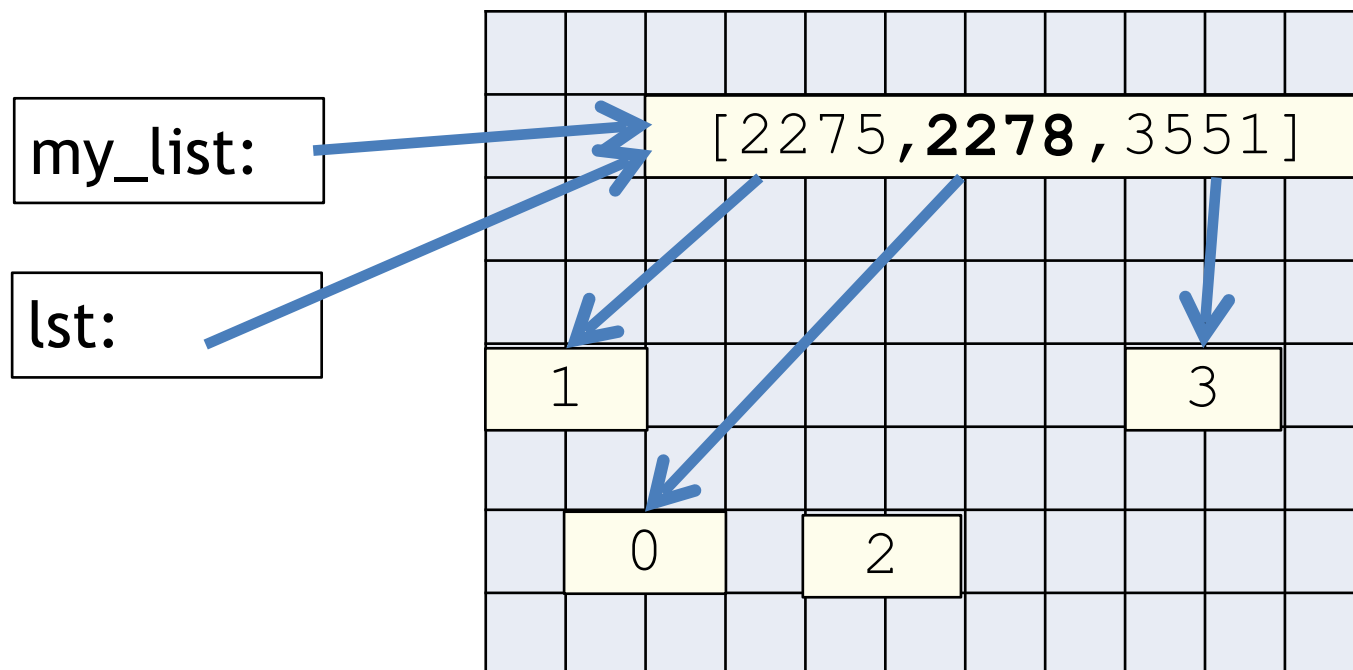


רשימות ופונקציות

- אפשר להעביר לפונקציה רשימה כפרמטר:

```
def set_zero(lst, i):  
    lst[i] = 0
```

```
my_list = [1, 2, 3]  
set_zero(my_list, 1)  
print(my_list)
```



התוצאה: הפונקציה משנה את הרשימה שהועברה לפרמטר lst, שתחום ההגדרה שלה הוא מחוץ לפונקציה!

דורש זהירות! יש לציין את זה בממשק של הפונקציה!

טיפוסים immutable

- אובייקט שהוא immutable לא ניתן לשינוי
(אבל משתנה שמצביע על אובייקט כזה כן ניתן לשינוי)
- אם זה container (למשל tuple), אז מראי המקום שהוא מכיל לא ניתנים לשינוי, אבל הם בעצמם יכולים להצביע על אובייקטים שהם mutable.

טיפוסים immutable

- אובייקט שהוא immutable לא ניתן לשינוי
(אבל משתנה שמצביע על אובייקט כזה כן ניתן לשינוי)
- אם זה container (למשל tuple), אז מראי המקום שהוא מכיל לא ניתנים לשינוי, אבל הם בעצמם יכולים להצביע על אובייקטים שהם mutable.

```
tpl = (15, 34, 65, 111)
```

```
lst = [1, 1, 2, 5, 14, 41, 132, 429]
```

```
tpl[2] = 68
```

```
lst[5] = 42
```

```
tpl = tpl + (175, )
```

```
lst.append(1430)
```

```
tpl.append(260)
```

טיפוסים immutable

- אובייקט שהוא immutable לא ניתן לשינוי
(אבל משתנה שמצביע על אובייקט כזה כן ניתן לשינוי)
- אם זה container (למשל tuple), אז מראי המקום שהוא מכיל לא ניתנים לשינוי, אבל הם בעצמם יכולים להצביע על אובייקטים שהם mutable.

```
tpl = (15, 34, 65, 111)
```

```
lst = [1, 1, 2, 5, 14, 41, 132, 429]
```

```
tpl[2] = 68 X
```

```
lst[5] = 42
```

```
tpl = tpl + (175, )
```

```
lst.append(1430)
```

```
tpl.append(260)
```

טיפוסים immutable

- אובייקט שהוא immutable לא ניתן לשינוי
(אבל משתנה שמצביע על אובייקט כזה כן ניתן לשינוי)
- אם זה container (למשל tuple), אז מראי המקום שהוא מכיל לא ניתנים לשינוי, אבל הם בעצמם יכולים להצביע על אובייקטים שהם mutable.

```
tpl = (15, 34, 65, 111)
```

```
lst = [1, 1, 2, 5, 14, 41, 132, 429]
```

```
tpl[2] = 68 ✗
```

```
lst[5] = 42 ✓
```

```
tpl = tpl + (175, )
```

```
lst.append(1430)
```

```
tpl.append(260)
```

טיפוסים immutable

- אובייקט שהוא immutable לא ניתן לשינוי
(אבל משתנה שמצביע על אובייקט כזה כן ניתן לשינוי)
- אם זה container (למשל tuple), אז מראי המקום שהוא מכיל לא ניתנים לשינוי, אבל הם בעצמם יכולים להצביע על אובייקטים שהם mutable.

```
tpl = (15, 34, 65, 111)
```

```
lst = [1, 1, 2, 5, 14, 41, 132, 429]
```

```
tpl[2] = 68 ✗
```

```
lst[5] = 42 ✓
```

```
tpl = tpl + (175, ) ✓
```

```
lst.append(1430)
```

```
tpl.append(260)
```

טיפוסים immutable

- אובייקט שהוא immutable לא ניתן לשינוי
(אבל משתנה שמצביע על אובייקט כזה כן ניתן לשינוי)
- אם זה container (למשל tuple), אז מראי המקום שהוא מכיל לא ניתנים לשינוי, אבל הם בעצמם יכולים להצביע על אובייקטים שהם mutable.

```
tpl = (15, 34, 65, 111)
```

```
lst = [1, 1, 2, 5, 14, 41, 132, 429]
```

```
tpl[2] = 68 ✗
```

```
lst[5] = 42 ✓
```

```
tpl = tpl + (175, ) ✓
```

```
lst.append(1430) ✓
```

```
tpl.append(260)
```

טיפוסים immutable

- אובייקט שהוא immutable לא ניתן לשינוי
(אבל משתנה שמצביע על אובייקט כזה כן ניתן לשינוי)
- אם זה container (למשל tuple), אז מראי המקום שהוא מכיל לא ניתנים לשינוי, אבל הם בעצמם יכולים להצביע על אובייקטים שהם mutable.

```
tpl = (15, 34, 65, 111)
```

```
lst = [1, 1, 2, 5, 14, 41, 132, 429]
```

```
tpl[2] = 68 ✗
```

```
lst[5] = 42 ✓
```

```
tpl = tpl + (175, ) ✓
```

```
lst.append(1430) ✓
```

```
tpl.append(260) ✗
```

הדגמה עם str

```
x = "hi"  
y = "bye"  
  
z = x + y      # z is a new string!  
  
w = z[2:4]     # slicing creates a new string!  
  
x[0] = 3  
x.append("a")
```

הדגמה עם str

```
x = "hi"  
y = "bye"  
  
z = x + y      # z is a new string!  
  
w = z[2:4]     # slicing creates a new string!  
  
x[0] = 3 X  
x.append("a")
```

הדגמה עם str

```
x = "hi"  
y = "bye"  
  
z = x + y      # z is a new string!  
  
w = z[2:4]     # slicing creates a new string!
```

```
x[0] = 3 X  
x.append("a") X
```

מיחזור אובייקטים immutable

```
In[54]: x = "hello"
In[55]: y = "he" + "llo"
In[56]: x is y
Out[56]: True
In[57]: x = (0, 1, 2)
In[58]: y = (0, ) + (1, 2)
In[59]: x is y
Out[59]: False
In[60]: x==y
Out[60]: True
```

מיחזור אובייקטים immutable

```
In[54]: x = "hello"
```

```
In[55]: y = "he" + "llo"
```

```
In[56]: x is y
```

```
Out[56]: True
```

לעיתים ממחזרים את

תוכן הזיכרון

```
In[57]: x = (0, 1, 2)
```

```
In[58]: y = (0, ) + (1, 2)
```

```
In[59]: x is y
```

```
Out[59]: False
```

```
In[60]: x==y
```

```
Out[60]: True
```

מיחזור אובייקטים immutable

```
In[54]: x = "hello"
```

```
In[55]: y = "he" + "llo"
```

```
In[56]: x is y
```

```
Out[56]: True
```

לעיתים ממחזרים את
תוכן הזיכרון

```
In[57]: x = (0, 1, 2)
```

```
In[58]: y = (0, ) + (1, 2)
```

```
In[59]: x is y
```

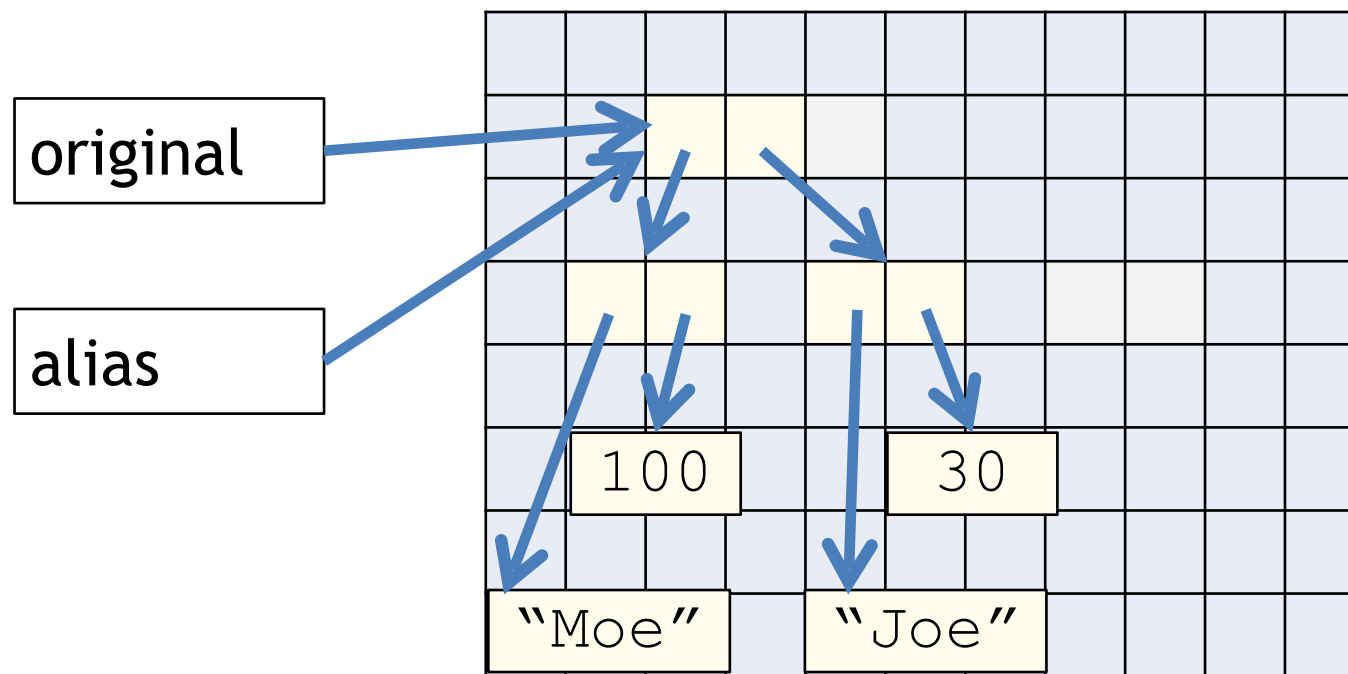
```
Out[59]: False
```

ולעיתים לא

```
In[60]: x==y
```

```
Out[60]: True
```

יצירת alias ל-list



```
original = ["Moe", 100], ["Joe", 30]
```

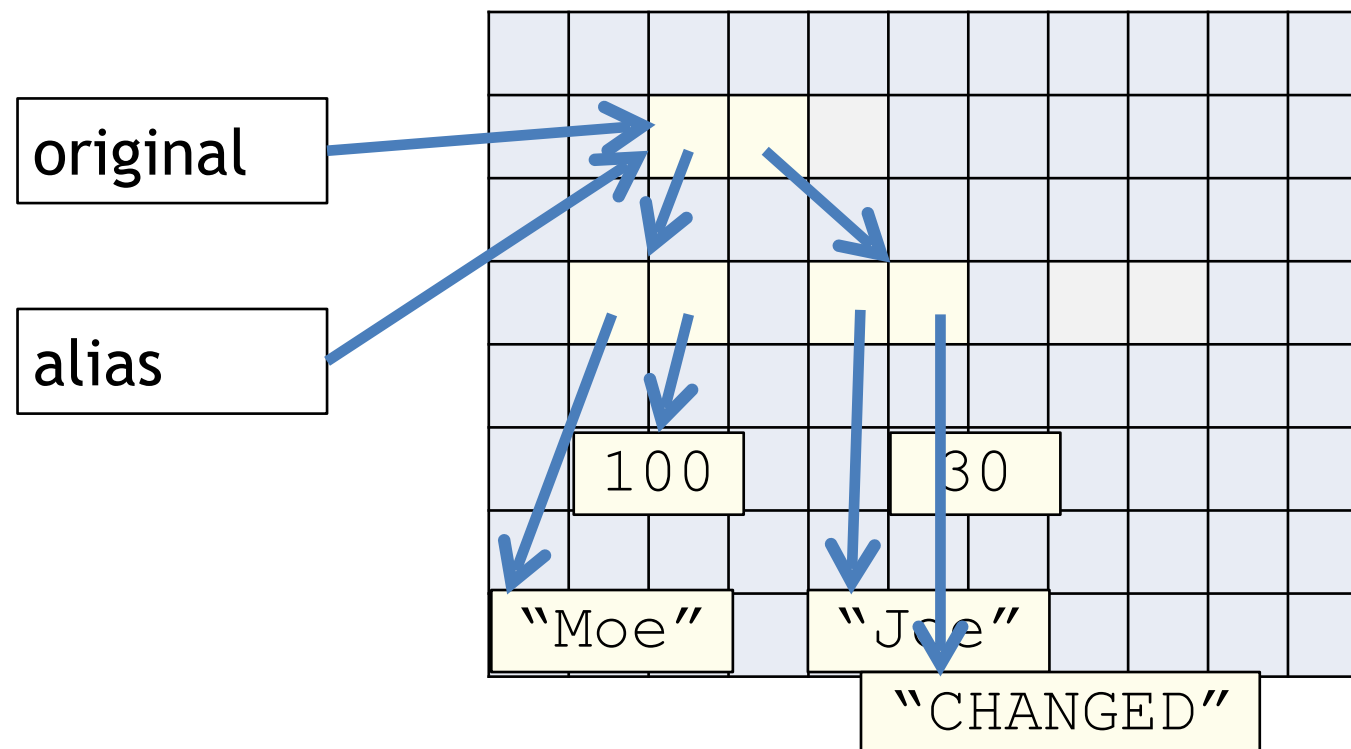
```
alias = original
```

```
original[1][1] = "CHANGED"
```

```
original.append(["Fred", 95])
```

```
print(alias)
```

יצירת alias ל-list



```
original = ["Moe", 100], ["Joe", 30]
```

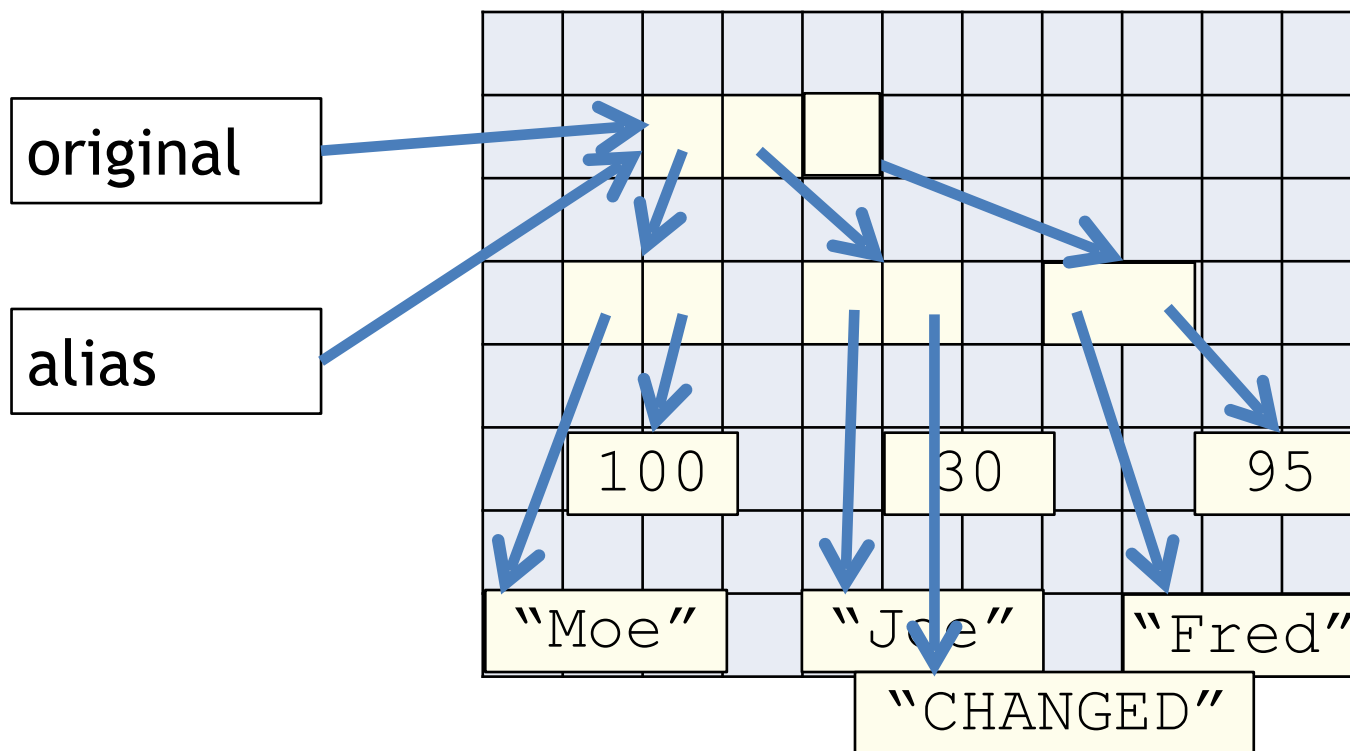
```
alias = original
```

```
original[1][1] = "CHANGED"
```

```
original.append(["Fred", 95])
```

```
print(alias)
```

יצירת alias ל-list



```
original = [ ["Moe", 100], ["Joe", 30] ]
```

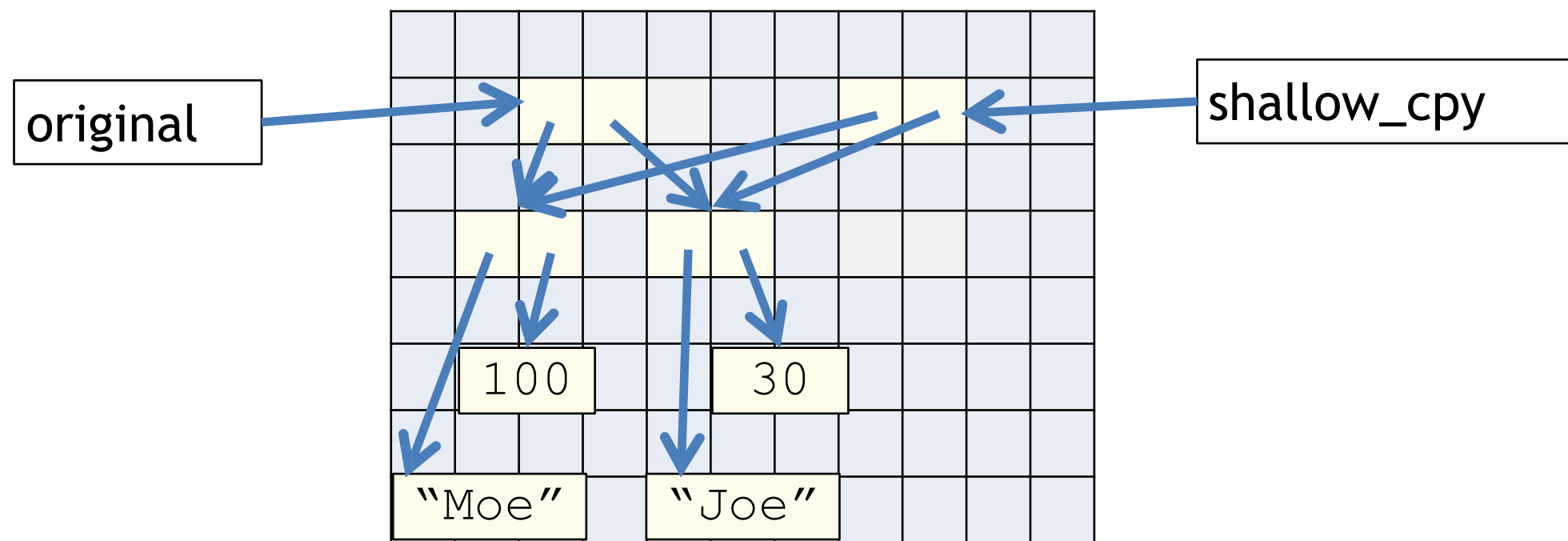
alias = original

```
original[1][1] = "CHANGED"
```

```
original.append(["Fred", 95])
```

```
print(alias)
```

list vs shallow copy

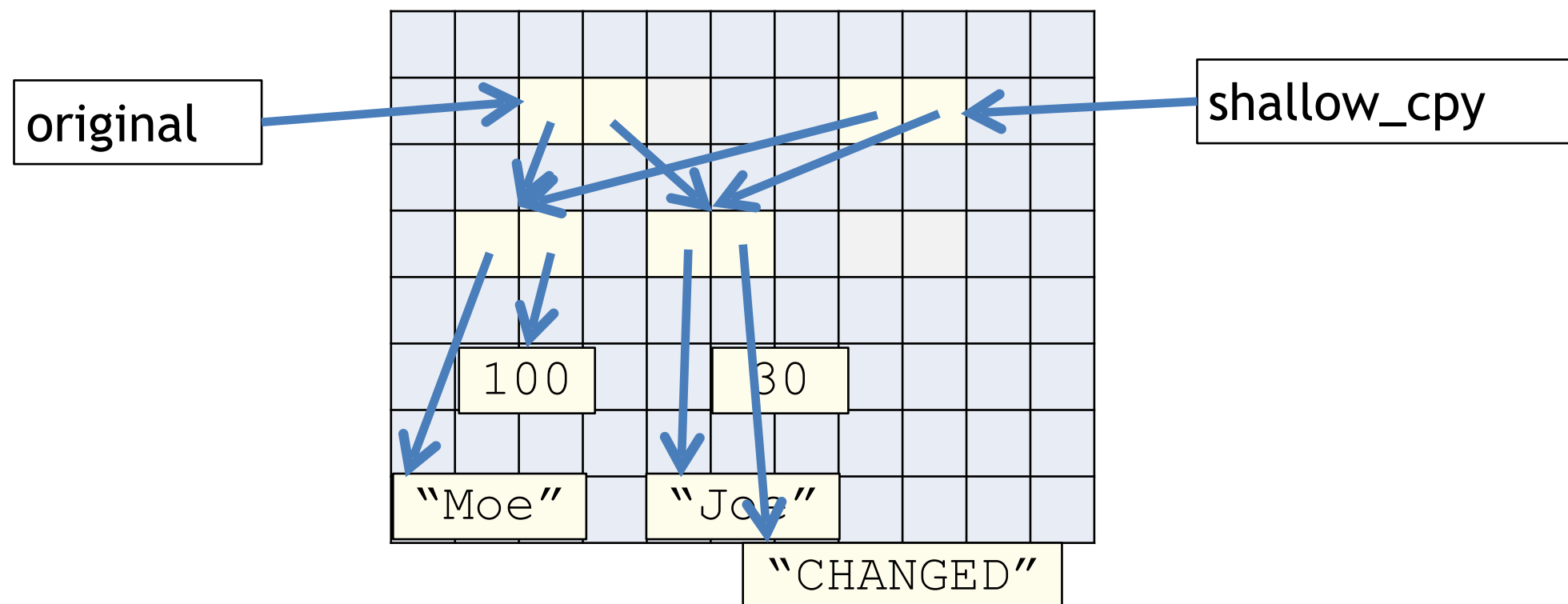


```
original = ["Moe", 100], ["Joe", 30]  
alias = original  
shallow_cpy = original[:]
```

```
original[1][1] = "CHANGED"  
original.append(["Fred", 95])
```

```
print(alias) ; print(shallow_cpy)
```

list vs shallow copy

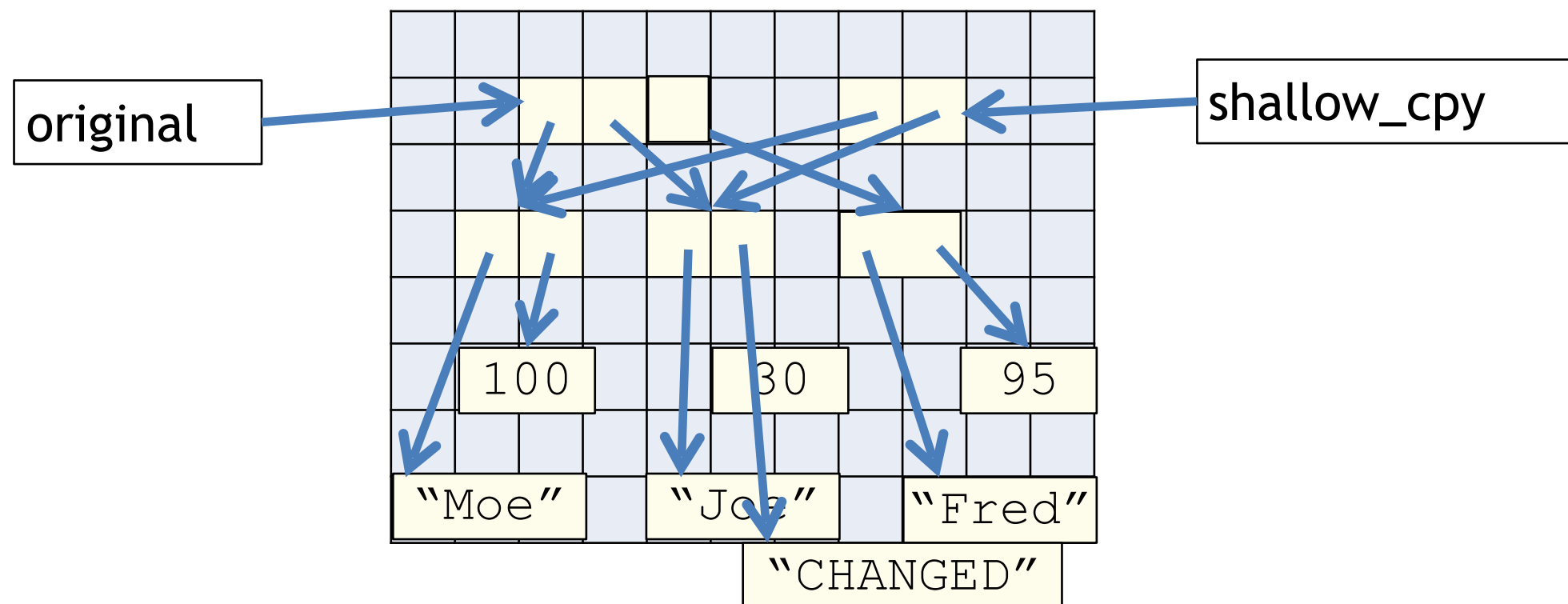


```
original = ["Moe", 100], ["Joe", 30]  
alias = original  
shallow_cpy = original[:]
```

```
original[1][1] = "CHANGED"  
original.append(["Fred", 95])
```

```
print(alias) ; print(shallow_cpy)
```

list vs shallow copy

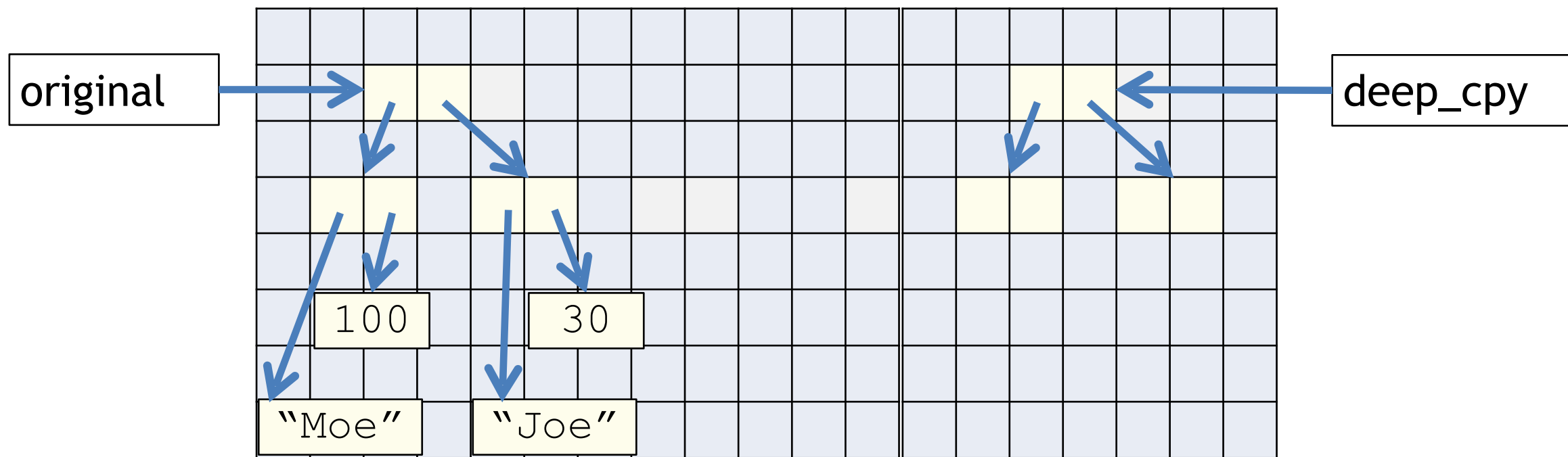


```
original = ["Moe", 100], ["Joe", 30]]  
alias = original  
shallow_cpy = original[:]
```

```
original[1][1] = "CHANGED"  
original.append(["Fred", 95])
```

```
print(alias) ; print(shallow_cpy)
```

list vs deep copy

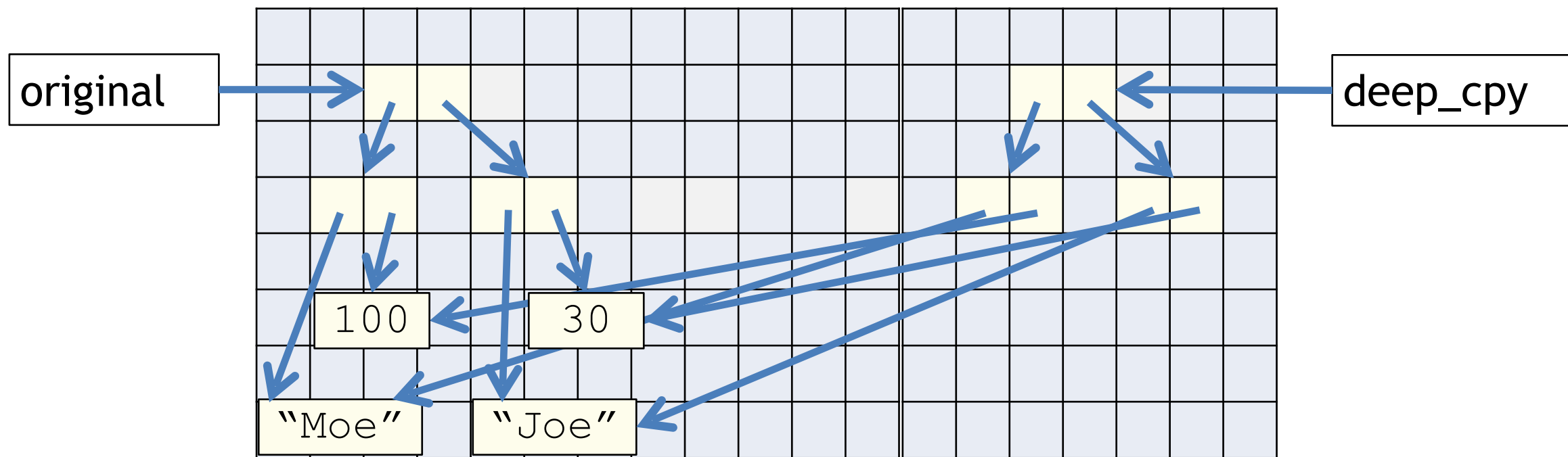


```
original = ["Moe", 100], ["Joe", 30]
alias = original
shallow_cpy = original[:]
deep_cpy = copy.deepcopy(original)

original[1][1] = "CHANGED"
original.append(["Fred", 95])

print(alias) ; print(shallow_cpy) ; print(deep_cpy)
```

list vs deep copy

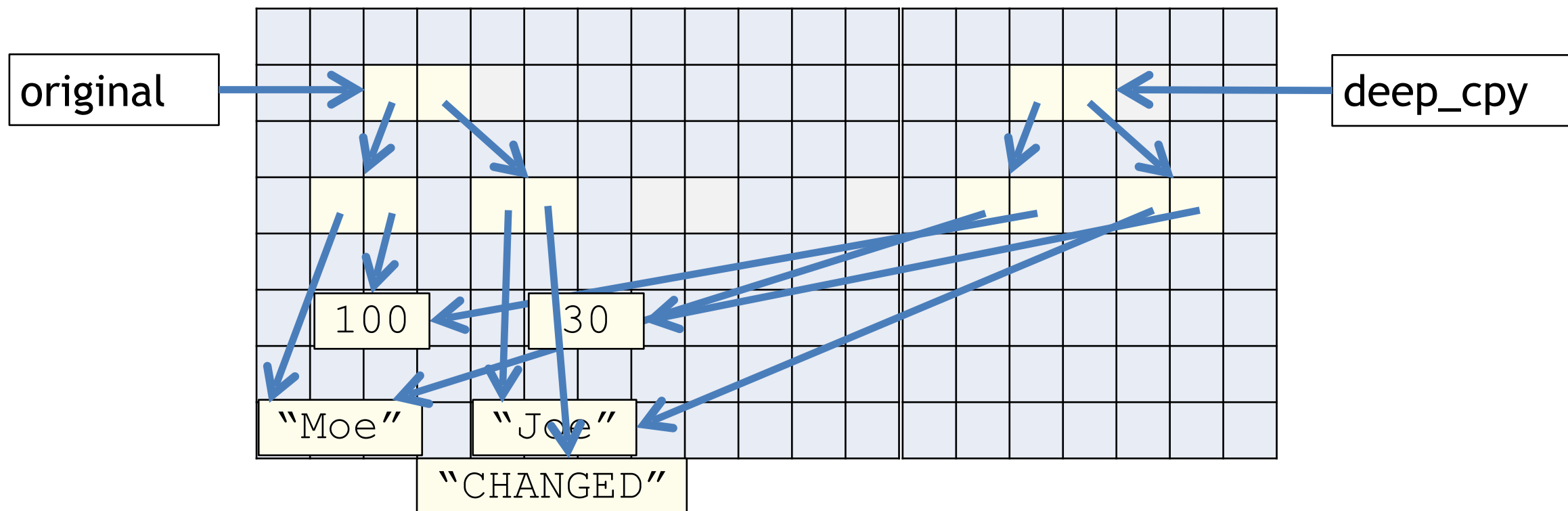


```
original = [["Moe", 100], ["Joe", 30]]
alias = original
shallow_cpy = original[:]
deep_cpy = copy.deepcopy(original)

original[1][1] = "CHANGED"
original.append(["Fred", 95])

print(alias) ; print(shallow_cpy) ; print(deep_cpy)
```

list vs deep copy

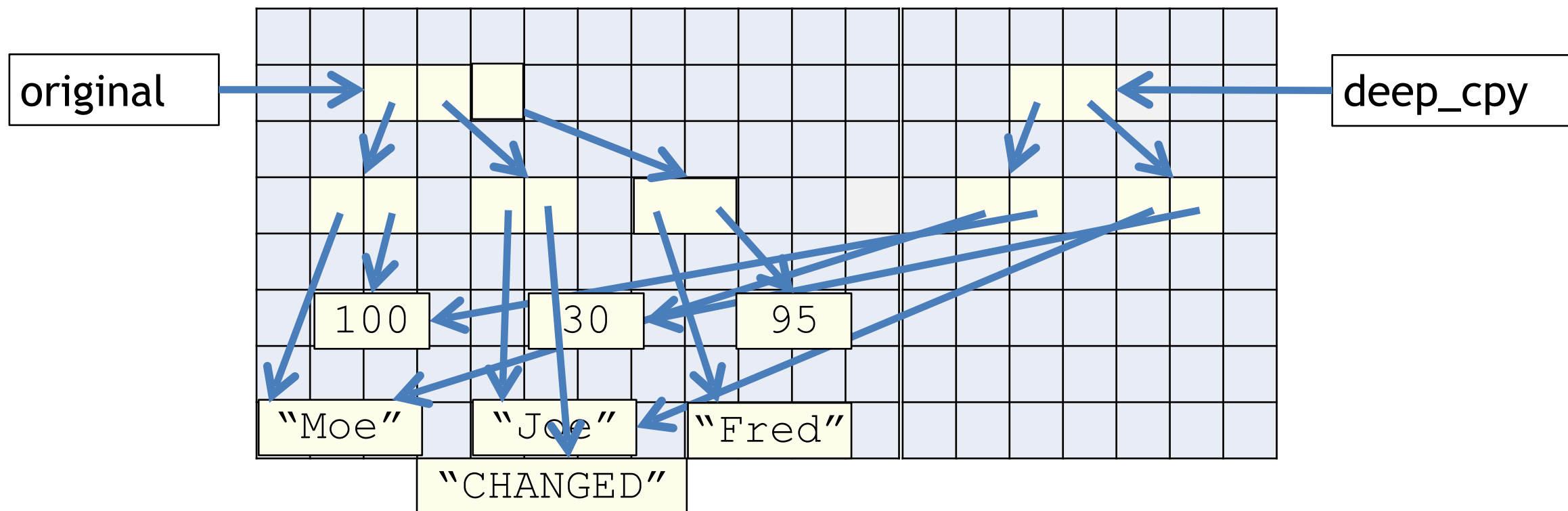


```
original = [["Moe", 100], ["Joe", 30]]
alias = original
shallow_cpy = original[:]
deep_cpy = copy.deepcopy(original)

original[1][1] = "CHANGED"
original.append(["Fred", 95])

print(alias) ; print(shallow_cpy) ; print(deep_cpy)
```

list vs deep copy



```
original = [["Moe", 100], ["Joe", 30]]
alias = original
shallow_cpy = original[:]
deep_cpy = copy.deepcopy(original)

original[1][1] = "CHANGED"
original.append(["Fred", 95])

print(alias) ; print(shallow_cpy) ; print(deep_cpy)
```

השוואת רשימות

- אופרטור ההשוואה == מבצע השוואה עמוקה בין רשימות
- המימוש בודק שאברי הרשימות מכילים אותם ערכים

```
original = [["Moe", 100], ["Joe", 30]]  
deep_cpy = copy.deepcopy(original)  
  
print(deep_cpy is original)           #False  
print(deep_cpy[0] is original[0])     #False  
print(deep_cpy == original)           #True
```

Mind benders

- The variable **a** is a tuple reference.
- Tuples are immutable.

```
a = ([1, 2, 3], "hi", 2.0)
a[0][0] = 52
print(a)
```

- Will this code terminate without error?
- Did we change the contents of the tuple?
- What will be printed?

Mind benders

- What does the following code do?

```
a = [1, 2]
b = [1, 2]
a[0] = b
b[0] = a

print(a[0])
```

- Can a list contain itself?